THE UNIVERSITY OF DANANG

VN-UK INSTITUTE FOR RESEARCH AND EXECUTIVE EDUCATION





DEVELOPMENT OF A CARO WEB APPLICATION USING ADVERSARIAL SEARCH ALGORITHMS

BASIC PROJECT REPORT

Student: Nguyễn Tâm Đức

Student ID: 24020005

Supervisor: Nguyễn Văn Thọ

Class: CSE24

Academic year: 2026–2027

ABSTRACT

This project studies move selection in Caro as a two-player adversarial decision problem on a 15 × 15 board under Freestyle rules. The algorithms are implemented in a Rust/WebAssembly engine, and a React/PixiJS application executes search in a Web Worker to examine the connection between a computed decision and a consistent game state. The report explains board representation, legal moves, win detection, heuristic evaluation, candidate selection, Minimax and Alpha-Beta pruning. It also examines hashing, transposition tables, quiescence and iterative deepening in separate reference and legacy implementations. The recorded native engine experiment contains 816 search measurements, including 408 paired Minimax and Alpha-Beta comparisons and 820 state-restoration checks. Separate records examine agreement between the JavaScript adapter and the WebAssembly binary, browser Worker responses and selected interface interactions. The reference experiments show that fewer visited nodes can coincide with longer execution time when move ranking adds work. They also identify immediate tactics lost by a hard candidate cap and an incorrect score caused by reusing cached entries after the root player changes. These results connect search algorithms to an interactive application and show why candidate policy, state restoration and cache context must be specified alongside search depth.

Keywords: Caro; adversarial search; Alpha-Beta pruning; selective search; WebAssembly.

CONTENTS

ABSTRACT....1
LIST OF FIGURES....5
LIST OF TABLES....6
ABBREVIATIONS AND READING CONVENTIONS....7
CHAPTER 1. INTRODUCTION....1
1.1 Background and motivation....1
1.2 Objectives and research questions....1
1.3 Scope, versions and source identity....2
1.4 Method and report structure....3
CHAPTER 2. ALGORITHMS AND DATA STRUCTURES....4
2.1 Scope, evidence, and notation....4
2.2 Board representation and legal updates....5
2.2.1 Matrix, index conversion, and state ownership....5
2.2.2 Apply and undo....6
2.3 Freestyle win detection....8
2.4 Baselines: random play and rule-based play....10
2.4.1 Random baseline....10
2.4.2 Immediate tactics and one-ply heuristic....11
2.5 Static evaluation and terminal utility....13
2.5.1 Evaluation of nonterminal positions....13
2.5.2 Terminal states and the score hierarchy....15
2.6 Candidate generation and a soft tactical cap....16
2.6.1 Spatial neighborhood....16
2.6.2 Tactical preservation and membership....18
2.7 Move ordering versus move selection....19
2.8 Minimax search....21
2.8.1 Recurrence and contract....21
2.8.2 Proof, example and complexity....22
2.9 Alpha-Beta pruning....24
2.9.1 Bounds and pseudocode....24
2.9.2 Worked cutoff and correctness boundary....25
2.10 Iterative deepening and budgets....26
2.11 Zobrist hashing and incremental state....28
2.11.1 Definition and update....28
2.12 Transposition table: entries, bounds and context....30
2.12.1 Meaning of stored values....30
2.12.2 Why reuse can be wrong....31

2.13 Quiescence search in the legacy TypeScript bot..... 33
2.14 Legacy bitmask patterns and incremental line counts.....35
2.15 Difficulty configurations and the active request path..... 38
2.16 Synthesis and limits of the algorithmic claims.....40
CHAPTER 3. SYSTEM ANALYSIS AND DESIGN.....42
3.1 Decision problem and algorithmic requirements..... 42
3.2 Non-functional requirements and acceptance boundaries.....45
3.3 Architecture and ownership..... 45
3.4 Board state, history and legal transition contract..... 47
3.5 Match phases and state transitions..... 47
3.6 Asynchronous request protocol and cancellation.....48
3.7 Coordinate transformation and gesture interpretation.....49
3.8 Search-result contract and presentation of telemetry..... 50
3.9 Verification scenarios derived from the design..... 51
CHAPTER 4. IMPLEMENTATION..... 53
4.1 Toolchain and project organization..... 53
4.2 Accepted move processing.....53
4.3 AI dispatch, Worker execution and WASM hydration.....54
4.4 Board rendering and interface synchronization..... 56
4.5 Presentation website and interactive explanation.....56
4.6 Python/Pyodide execution and four instructional programs..... 57
4.6.1 Pattern scanner.....57
4.6.2 Heuristic calculator.....58
4.6.3 Chebyshev candidate example.....58
4.6.4 Empirical-analysis snippet.....59
4.7 Implementation traceability and limits.....60
CHAPTER 5. RESULTS AND EVALUATION.....61
5.1 Evaluation questions, evidence and boundaries.....61
5.2 Native board and search checks of the 23 October engine..... 61
5.3 WebAssembly parity and browser observations..... 63
5.3.1 Follow-up application validation.....64
5.4 Separate native reference experiments.....64
5.4.1 A paired Minimax and Alpha-Beta example.....65
5.4.2 Candidate caps, ordering and tactical completeness.....66
5.4.3 Hash, table scope and interrupted search.....67
5.5 Historical cross-language and Week 4 datasets.....67
5.6 Presentation examples and evidence quality.....69
5.7 Threats to validity and interpretation..... 69

CHAPTER 6. CONCLUSION AND FUTURE WORK..... 71
6.1 Work completed and contribution..... 71
6.2 Limitations..... 72
6.3 Future development..... 72
REFERENCES..... 74
APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP..... 82
APPENDIX B. EVIDENCE AND VERIFICATION REGISTER..... 84
APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE..... 87

LIST OF FIGURES

Figure 2.1. Row-major board representation 6
Figure 2.2. Search must restore its caller state 8
Figure 2.3. Last-move scan along four axes 10
Figure 2.4. Rule-based tactical priorities 13
Figure 2.5. Contiguous-run weights of the archived Rust evaluator 15
Figure 2.6. Radius and board edges affect candidate membership 18
Figure 2.7. Alternative traversal orders on identical membership 21
Figure 2.8. A two-level Minimax example 23
Figure 2.9. Alpha-Beta cutoff on an unchanged tree 26
Figure 2.10. Iterative deepening and incomplete work 28
Figure 2.11. Incremental XOR and turn-key handling 30
Figure 2.12. Bound semantics in a transposition table 33
Figure 3.1. Responsibilities in the playable application 46
Figure 3.2. Match phases and the separate AI waiting flag 48
Figure 3.3. Inverse camera transform before board bounds checking 50
Figure 4.1. Tactical and Rust routes converge at the guarded commit 55
Figure 4.2. Union of clipped Chebyshev neighborhoods 59
Figure 5.1. Recorded native engine nodes on the center fixture 63
Figure 5.2. Native reference node counts (ranking cost excluded) 66
Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture 68

LIST OF TABLES

Table 2.1. Recorded Rust difficulty configurations and frontend request path 39
Table 3.1. Algorithmic requirements AR01–AR09 42
Table 3.2. Verification scenarios and expected outcomes 51
Table 5.1. Native engine search on the center fixture at depth two 62
Table 5.2. Native reference search on the clustered fixture at depth two 65
Table 5.3. Historical native Rust and TypeScript/Node benchmark 68
Table A.1. Repository inventory and roles 82
Table B.1. Evidence summary and verification status 84

ABBREVIATIONS AND READING CONVENTIONS

AI — Artificial Intelligence.

DSA — Data structures and algorithms.

DFS — Depth-first search.

DOM — Document Object Model.

PRNG — Pseudorandom number generator.

PvP — Player versus player; two people share one application.

PvE — Player versus environment; in this project, one person plays against the computer.

TT — Transposition table.

UI — User interface.

WASM — WebAssembly.

Ply — One stone-placement move by one side.

Fixture — A deliberately prepared input position for a test or measurement.

Coordinates — Frontend (row,col) equals Rust (y,x); indices run from 0 to 14.

Evidence — Source code explains implementation mechanisms; experiment records provide results for the stated inputs and environment.

CHAPTER 1. INTRODUCTION

1.1 Background and motivation

Caro is a two-player placement game in which a small set of rules produces a large adversarial decision space. A player places one stone on an empty cell, attempts to complete a winning line and must anticipate the opponent's reply. The board representation is simple enough to implement explicitly, while the number of possible move sequences makes exhaustive search impractical beyond shallow depths. This combination makes the project suitable for studying data structures, recursive search, evaluation functions, pruning and the relationship between algorithmic results and an interactive application.

The project includes a playable web application and a separate website for explaining its algorithms. React provides the interface, PixiJS renders the board, and GameController coordinates accepted moves, match state and AI requests. Move selection combines a TypeScript tactical check with a Rust/WebAssembly search executed in a Worker. The repository also contains an earlier TypeScript search module, native benchmark programs and weekly project reports. The application, benchmarks and teaching examples share the game domain but execute through different entry points [1], [2], [3], [4], [5], [6], [7].

The practical problem is to select a legal move while preserving the state of an interactive match. Search must leave its input board unchanged, follow the selected win rule and return a result that belongs to the active round. Reset, Undo and asynchronous computation make these requirements part of the algorithm integration. The report therefore studies move quality together with the controller checks that determine whether an AI response can be committed.

1.2 Objectives and research questions

The first objective is to formulate Caro as an adversarial decision problem: a board state, legal stone placements, alternating players and a terminal outcome. The application uses a 15 × 15 board and the Freestyle win condition, under which a contiguous line of at least five stones wins, including longer lines. The report derives the last-move detection procedure and states the assumptions that make it sufficient. The second objective is to explain how evaluation, candidate selection and recursive search turn a position into a move, with worked examples, correctness arguments and computational complexity [8].

The third objective is to examine the conditions under which a computed move remains a valid decision in an asynchronous application. Board ownership, guarded move commitment, history replay into WebAssembly and stale-response rejection connect algorithm inputs and outputs to a consistent game state. The fourth objective is experimental: compare Minimax and Alpha-Beta on the same candidate tree, separate move-ranking work from recursive search visits, and test whether selective candidate policies retain immediate winning and blocking moves.

Four research questions guide the report. RQ1 concerns deterministic board updates and Freestyle win detection. RQ2 concerns how heuristic evaluation, candidate selection and adversarial recursion produce a move. RQ3 examines the effects of pruning, ordering and caching on search results and computational work. RQ4 concerns state consistency when native search components are used through an asynchronous browser interface. Chapter 5 relates each question to the corresponding implementation and experimental records.

1.3 Scope, versions and source identity

The published source reference is tducn110/basicproject at revision 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. CaroGame contains the application and its WebAssembly package; present contains the teaching website. The weekly folders, benchmark and research contain documentation and experimental records. The Rust baseline in research/current_2026_10_23/baseline_engine is an earlier implementation; the modified engine is identified by hashes and selected excerpts [1], [9], [10], [11]. A local follow-up on 13 November 2026 includes that modified source in CaroGame/engine and rebuilds the application package from it. Appendix B records this release separately from the pinned experiments, and Appendix C gives both reproduction paths.

Four implementations are distinguished throughout the analysis: the Rust/WebAssembly engine used by the application, the archived Rust baseline, the legacy TypeScript bot, and the native Rust reference library used by the separate research harness. An algorithm or measurement is attributed to the implementation that supplies it [10], [12], [13].

The study covers board representation, Freestyle win detection, heuristic evaluation, selective candidate generation and adversarial search, together with the state and execution contracts needed to use their results correctly. Local two-player play, play against the computer and interactive demonstrations provide the application context for that study. Online multiplayer, learned policies and model training are outside

the project. Chapter 5 evaluates the recorded algorithms and their integration under the stated experimental conditions; Chapter 6 discusses the limits of those results and further development.

1.4 Method and report structure

The method combines source inspection with analysis of recorded experiments. The implementation trace follows pointer input, coordinate conversion, move validation, board updates, terminal-state checks, AI dispatch, response validation and display updates. Search comparisons keep the rule, evaluation weights, depth and candidate policy fixed where equivalence is being tested. Primary literature supplies the theoretical basis for search and hashing, while the project references identify the code and datasets used in the analysis.

Chapter 2 presents the data structures and algorithms. Chapter 3 explains the requirements, component responsibilities and state transitions. Chapter 4 describes the application and teaching website. Chapter 5 analyzes the native experiments, WebAssembly adapter checks, browser observations and benchmark datasets. Chapter 6 summarizes the contribution, limitations and future work. The appendices provide the repository inventory, a summary of experimental records and instructions for inspecting and building the published snapshot.

CHAPTER 2. ALGORITHMS AND DATA STRUCTURES

2.1 Scope, evidence, and notation

This chapter develops the board representation, terminal rule, evaluation function and search algorithms used in the Caro project. The current frontend and its Worker adapter are examined alongside two research implementations: the modified Rust engine recorded in the 23 October source snapshot [10], and the separate native reference engine [13]. The active application calls a vendored WebAssembly package through the request interface and Worker adapter [3], [6], [14]. These sources have different roles: frontend source establishes request and commit behavior, Rust source explains the recorded algorithm, and binary execution establishes the behavior of the tested package.

The repository also retains a TypeScript bot under src/game/bot. It is referred to as the legacy bot because AIController sends ordinary requests to GomokuWasmEngine and uses a separate tactical check for the four higher difficulty levels. Iterative deepening, quiescence search and transposition-table examples in this chapter come from that legacy implementation [3], [15]. Experiments on the separate Rust reference engine are identified as native reference results [13].

The notation throughout uses an N × N board with N = 15 and winning length K = 5 unless stated otherwise. Coordinates (x,y) mean column and row, respectively, with both in 0…14. In the frontend code the equivalent coordinate is (row,col) = (y,x). A ply is one placed stone. Let B denote a board, p the side to move, opp(p) the other side, and m a legal move. The transition T(B,p,m) places p at m; it is defined only if the cell is empty and the game permits a move. Let A(B,p) be the moves the selected search policy actually generates, which may be a strict subset of the legal empty cells. A root player R is the player whose advantage is represented by a larger score. These distinctions matter because a proof about the generated search tree cannot establish a claim about every legal move if the generation policy has already excluded some moves.

The formulations and pseudocode state the input, transition rule and assumptions of each algorithm. Worked examples illustrate their operation; measurements are presented separately in Chapter 5. Depth-limited search follows the distinction between look-ahead and static horizon evaluation described by Shannon [16].

2.2 Board representation and legal updates

2.2.1 Matrix, index conversion, and state ownership

A position contains 225 cells, each empty, X or O. For a logical matrix B[y][x], row-major conversion is i = 15y + x, with inverse x = i mod 15 and y = floor(i/15). The center (7,7) therefore has index 112, and (14,14) has index 224. In the modified Rust source, Board stores a 15 × 15 array of repr(u8) cells and a move counter. Its cell values are Empty = 0, Black = 1 and White = 2; the array occupies 225 bytes, while the complete Board also contains the counter and any alignment padding [10], [17]. The frontend GameState instead stores occupied coordinates in a Map and maintains ordered move history [18]. The legacy BotBoard uses a one-dimensional array padded with boundary sentinels. These representations support different operations: direct Rust cell access, frontend match history, and incremental line analysis in the legacy bot.

Index conversion must preserve both the cell and its coordinate convention. For $0 \leq x,y < N$, division of $i = Ny + x$ by N has quotient y and remainder x, so the inverse returns the original coordinate. Conversely, every integer $0 \leq i < N^2$ determines exactly one such pair. This establishes a one-to-one mapping, with O(1) conversion time and space. Bounds must be checked before using an index: on a 15-column board, the invalid coordinate (15,0) would otherwise alias the valid coordinate (0,1). The Rust (x,y) and frontend (row,col) forms therefore require an explicit swap at their boundary.

In the modified Rust snapshot, Board::place checks bounds, rejects an empty stone or occupied cell, writes the stone and increments move_count. Board::undo rejects an invalid or empty coordinate, then clears the occupied cell and decrements the counter [10]. These operations maintain the position. Turn alternation, move history and terminal lockout belong to the Rust Game wrapper and the frontend match controller, rather than to Board::place itself [2].

Board mutation and match progression have separate owners. Board holds the cells and move count; Game and the frontend controller maintain turn, result and history. Search explores hypothetical moves on a working board, while the match commits only the selected move. The modified Rust search clones the root board once and applies and undoes moves within that copy [10]. This avoids a complete board copy for each child, although candidate generation still allocates move lists at internal nodes.

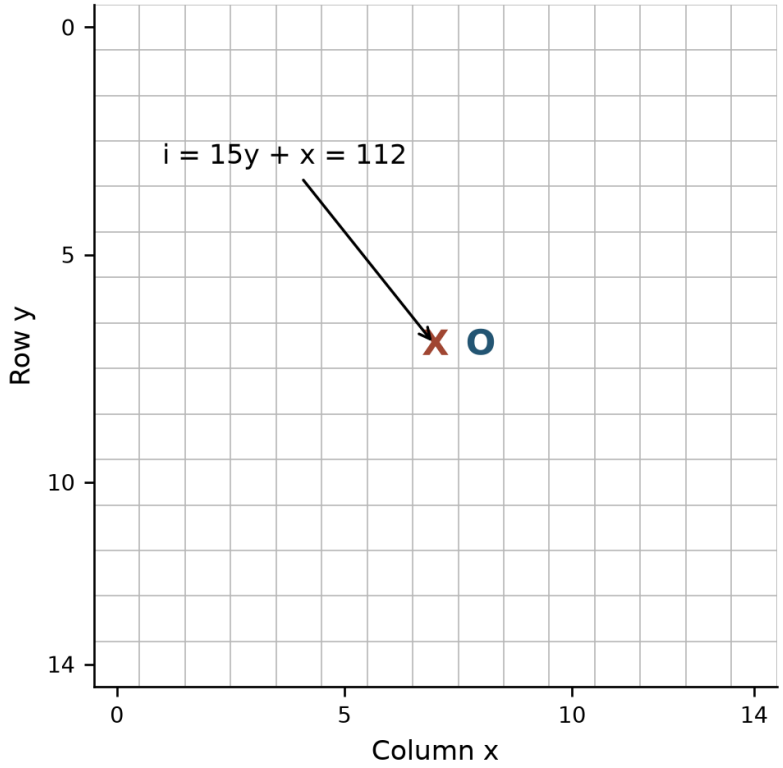


Figure 2.1. Row-major board representation

Source: Illustration of logical row-major indexing, based on the Rust board representation [10], [17].

2.2.2 Apply and undo

Inputs and output. APPLY(B,m,p) takes the working board, a coordinate and a nonempty player; it returns success or failure. UNDO(B,m) takes the coordinate of a previously applied move and returns whether removal was possible. The stronger search precondition is that undo is paired with the most recent successful hypothetical move on that branch. An individual undo that merely removes an occupied cell does not enforce the whole history discipline.

```
// Algorithm 2.1 (modified Rust source in [10]) — APPLY(B, (x,y), p):
    if x or y is outside the board, or p is empty: return failure
    if B[y][x] is not empty: return failure
    B[y][x] ← p
    B.move_count ← B.move_count + 1
    return success

UNDO(B, (x,y)):
    if (x,y) is outside the board or B[y][x] is empty: return failure
    B[y][x] ← empty
    B.move_count ← B.move_count - 1
    return success

VISIT_CHILD(B, p, m):
    require APPLY(B, m, p) succeeds
    try: value ← VISIT(B, opp(p), m)
    finally: require UNDO(B, m) succeeds
    return value
```

The try/finally structure in this explanatory pseudocode requires restoration after a return, cutoff or supported abort. An implementation must provide equivalent cleanup for every exit it supports. A matching apply/undo restores the cell and count in O(1) time; hashes, cached patterns, turn and other mutable fields must be restored separately by their owners. The modified Rust kernel uses structured returns without a deadline-abort path, whereas the legacy TypeScript search uses try/finally when it can throw a search-abort sentinel.

The requirement that apply and undo succeed is an internal precondition, not a recovery branch. In the modified Rust search, an illegal generated move reaches expect and fails the search; undo is checked by debug_assert, which is not a release-mode recovery mechanism. The ordinary search entry owns a root clone, but search_in_place exposes the caller's working board and assumes normal completion. Neither path establishes transactional recovery from an unexpected panic. In the legacy search, the dedicated deadline sentinel unwinds through paired finally blocks; unrelated exceptions are rethrown. A corrupted working state must not be interpreted as a valid search result. Recovery by discarding and reconstructing the request would require an explicit caller policy; it is not implemented by the pseudocode's require statement [10], [15].

Invariant and argument. Immediately before exploring each sibling, the working state must equal the state at the parent node. A successful apply changes exactly the selected empty cell and increments the count. Its matching undo clears that same

cell and decrements the count; therefore these fields equal their former values. By induction over nested calls, assuming each recursive child restores every state field it owns, the parent sees the same state for the next sibling. The induction depends on legal matching calls; it is not guaranteed if an exception bypasses undo or a different move is removed.

For example, let X occupy (7,7), O occupy (8,7), and move_count be two. Applying X at (7,8) sets that cell and raises the count to three; undoing the same move restores both fields. Clearing the cell without decrementing the counter would make fullness checks unreliable. With direct indexing, each cell/count update takes O(1) time and O(1) auxiliary space; copying an N × N board takes O(N²). The archived native experiment records 820 successful working-board restoration checks, including repeated search configurations. This is the finite test result used in Chapter 5 [19].

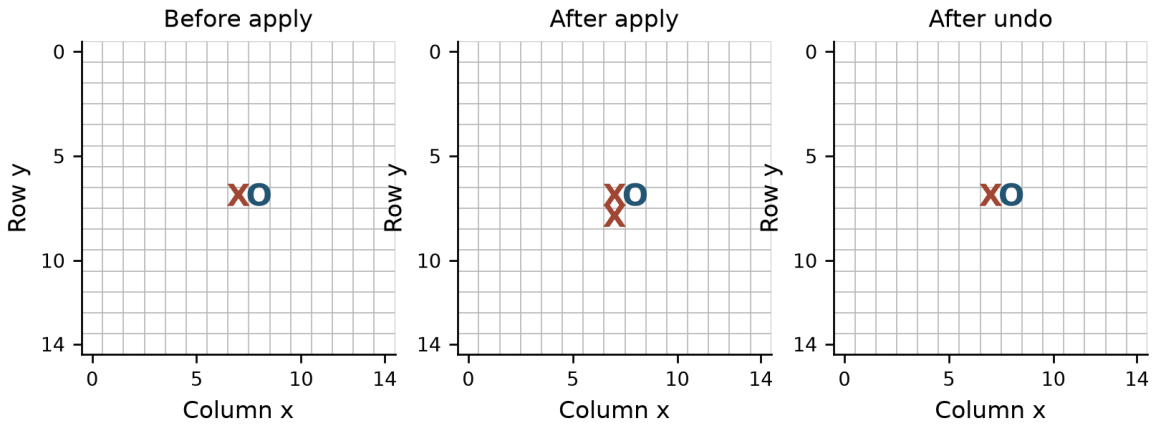


Figure 2.2. Search must restore its caller state

Source: Illustration of paired apply/undo in the modified Rust source [10].

2.3 Freestyle win detection

The frontend win rule treats a contiguous line of at least five stones as a win, including an overline of six or more. It checks four axes through the last move, with both directions on each axis. It does not require exactly five or impose an additional two-blocked-ends condition [8]. The term Caro is used for the application, but this particular rule is specified explicitly because games with similar names may use other winning conditions.

The procedure receives a valid board and the last placed coordinate m. It returns the winning player and collected winning cells, or no result. It assumes the preceding position was unfinished, so any newly created winning line passes through m. An

arbitrary imported position instead requires a full-board result check at the search root. Under the frontend state contract, an empty last-move coordinate yields no winner [8].

The reason to use the last move is local change: only one empty cell becomes occupied. Every line that does not pass through that cell has exactly the same stones as before, so it cannot become a new winning line if the preceding board had no winner. This argument justifies restricting the scan to four axes; it does not validate arbitrary imported boards or detect a pre-existing win elsewhere. At the root, the modified Rust search uses check_winner over the position before relying on check_winner_at for descendants [10].

For directions D = {(1,0), (0,1), (1,1), (1,-1)}, let f_d and b_d count consecutive stones equal to the stone at m in the positive and negative direction. A win through the last move occurs exactly when some 1 + f_d + b_d ≥ K. To decide whether the threshold is reached, scanning at most K−1 matching cells in each direction is sufficient: any additional cells cannot change a negative result to a positive result after K−1 matching cells already exist on one side.

```
// Algorithm 2.2 (CaroGame/src/game/core/WinRules.ts) — WIN_AT(B, m, K):
  if m is out of bounds or B[m] is empty: return NONE
  p ← B[m]
  for d in [(1,0),(0,1),(1,1),(1,-1)]:
    cells ← [m]
    for sign in [+1,-1]:
      for step from 1 to K-1:
        q ← m + sign * step * d
        if q is outside or B[q] ≠ p: break
        append q to cells
    if length(cells) ≥ K: return (p, cells)
  return NONE
```

The scan invariant is that every collected cell is on the selected axis, adjacent to the preceding segment and occupied by p. Stopping at the first boundary or different cell prevents joining disconnected sequences. If a contiguous run through m has length at least K, either its collected positive and negative portions together reach K−1 neighbors or a single portion does, so the check succeeds. Conversely, K collected stones form a contiguous same-player segment through m, establishing the stated rule. The returned display collection may be truncated to K−1 stones per side; it is sufficient to prove a win but is not necessarily the full maximal overline.

For X X [X] X X on a row, with the bracketed cell placed last, each side contributes two stones and 1 + 2 + 2 = 5. For X X X [X] X X, the run exceeds five and is still a win. A gap or board edge stops the corresponding scan. With K = 5, the frontend examines at most 4 × 2 × 4 = 32 neighboring positions, giving O(K) time and O(K) space for the collected cells. These bounds are constant for the configured winning length. A full-board scan checks every occupied starting point and has a different cost; the report does not infer an elapsed-time speedup from the operation count alone.

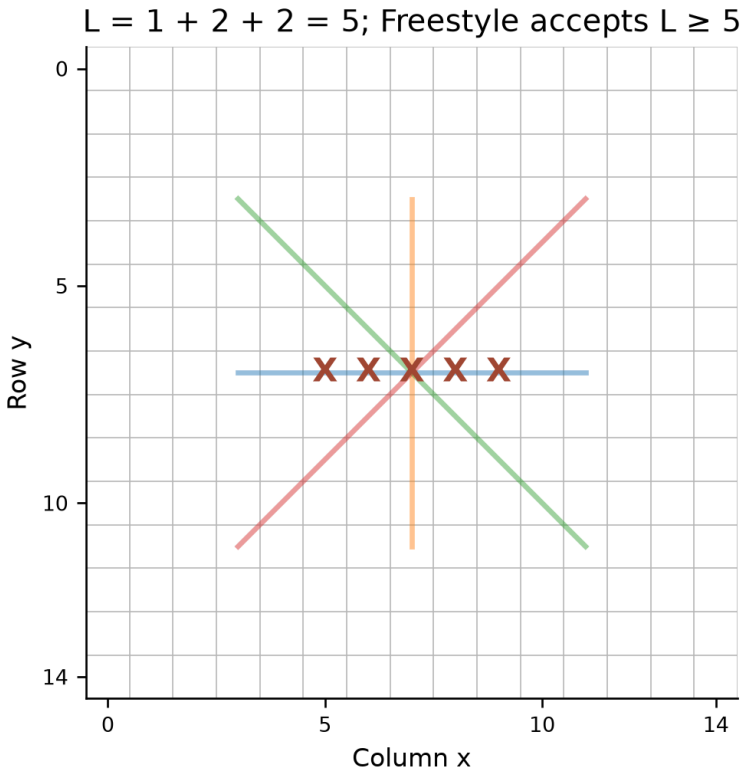


Figure 2.3. Last-move scan along four axes
Source: CaroGame/src/game/core/WinRules.ts [8].

2.4 Baselines: random play and rule-based play

2.4.1 Random baseline

A random baseline selects a legal move without comparing future replies. In the modified Rust snapshot, Easy first rejects an empty player or terminal board, enumerates empty cells, and selects an index using a seeded 64-bit mixer modulo the list length. It evaluates the chosen move afterwards to populate the result; that evaluation does not influence selection [10].

```
// Algorithm 2.3 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— RANDOM_MOVE(B, p, seed):
    if p is empty or B is terminal: return NONE
    L ← all legal empty cells in B
    if L is empty: return NONE
    i ← SEEDED_INTEGER(seed) mod length(L)
    return L[i]
```

The seed is part of the input because the mixer is deterministic. The output is a member of L or no move. A board with one stone has 224 empty cells, while a one-element list always returns its sole coordinate. Terminal handling precedes modulo selection, so a full board cannot cause division by zero. Index selection takes O(1) after enumeration. Modulo reduction does not in general produce an exactly uniform distribution over arbitrary list lengths.

For example, if the ordered legal list is [a,b,c] and the generated unsigned integer is 8, the remainder is 2 and the selected move is c. No comparison of the positions after a, b and c occurs. More generally, write $2^{64} = qL + r$ for a list of L cells. Under a hypothetical uniform 64-bit input, the first r remainders have q+1 preimages and the rest have q, explaining the modulo bias. This counting argument does not establish that a fixed seed is random or that a finite run samples moves uniformly. The source uses a SplitMix64-style mixer with wrapping 64-bit arithmetic [10].

Gathering all cells takes $O(N^2)$ time and $O(N^2)$ space in the worst case; selection after that is O(1). A post-selection evaluation has its own cost. A fixed seed makes the selection reproducible under the same generator and ordering, not necessarily across different implementations or changed inputs. A random policy does not promise to notice an immediate win or threat.

2.4.2 Immediate tactics and one-ply heuristic

The modified Rust Normal policy selects an immediate win when one is available, then an immediate blocking coordinate, and otherwise the highest-valued one-ply candidate. Candidate assessment supplies the win and block markers [10]. The frontend has a separate tactical resolver for Hard, Very Hard, Expert and Master; Easy and Normal bypass it. A frontend tactical result identifies its source as tactical and reports zero Rust nodes, because it returns before a WASM request [3], [20].

```
// Algorithm 2.4 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— RULE_BASED(B, p, policy):
    if B is terminal or p is invalid: return NONE
    C ← SELECT_CANDIDATES(B, p, policy)
    if an m in C wins immediately for p: return m
    if an m in C is an immediate winning cell for opp(p): return m
    best_move ← NONE; best_score ← -infinity
    for m in C:
        APPLY(B,m,p)
        try:
            score ← TERMINAL_OR_EVALUATE(B,p,m)
        finally:
            UNDO(B,m)
        if best_move is NONE or score > best_score:
            best_move ← m; best_score ← score
    return best_move, best_score
```

The decision invariant after processing the first j non-tactical candidates is that best_score is the greatest observed value among those j moves and the board is restored. A separate invariant places the win and block checks before the one-ply loop. Thus the routine picks the best score among the evaluated candidates when neither tactical condition applies. Correctness of the tactical marker itself depends on the assessment recognizing the corresponding winning geometry and on the move being included in C.

For a concrete threat, suppose X occupies row 0, columns 0–3, column 4 is empty, and O is to move. The left endpoint lies outside the board; placing O at (4,0) blocks the only immediate completion along this row. If the same four consecutive X stones are in the interior with both endpoints empty, there are two distinct winning cells. A single O move can occupy one cell, not both. Retaining both choices avoids losing information but does not establish that either saves the game. Similarly, an immediate winning move for O takes priority over blocking if both are available under the algorithm's ordering.

If k candidates are generated, one-ply evaluation costs roughly O(k × evaluation-cost), in addition to candidate generation and ordering. A constant-time local win scan does not make the whole rule-based procedure constant time. A nonterminal board with no generated move requires a defined fallback or no-move response; terminal handling must precede any selection. A short tactical prepass is not a substitute for searching through arbitrary combinations.

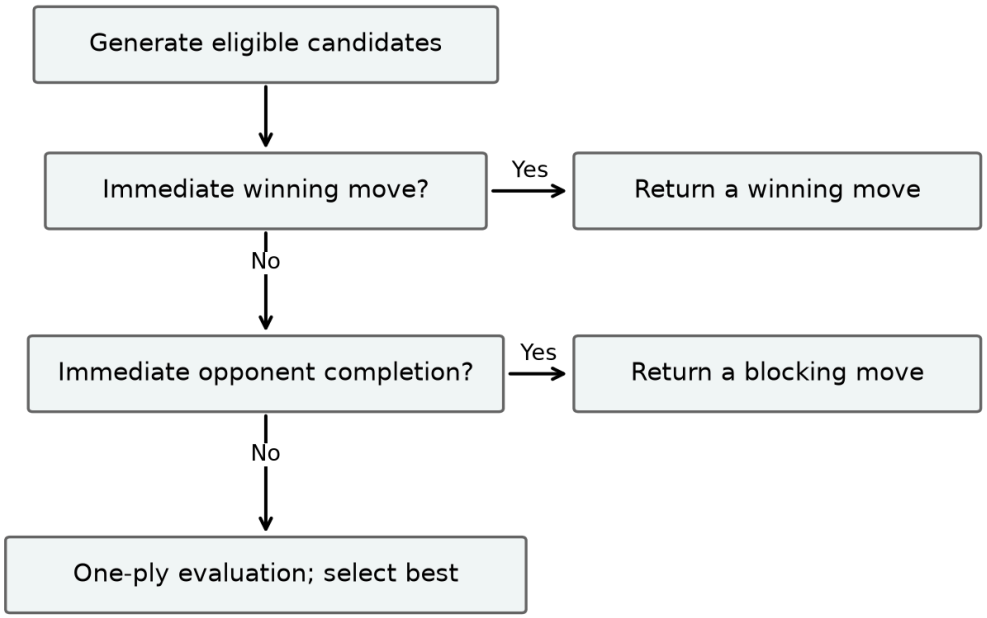


Figure 2.4. Rule-based tactical priorities

Source: Illustration of the modified Rust rule-based priorities [10].

2.5 Static evaluation and terminal utility

2.5.1 Evaluation of nonterminal positions

A search with finite depth can stop before a winner exists. Its evaluator assigns an approximate score to that horizon position. In the documented modified Rust snapshot, evaluate scans maximal contiguous runs in four directions, adds the root player's run weights, subtracts the opponent's, and adds or subtracts a center preference. The snapshot counts a run once per axis by skipping a cell when the previous cell on the same axis has the same color. It then clamps the final heuristic to a range below terminal scores [10].

Let $R_p(B)$ be maximal runs of p over the four axes, len(r) their length and open(r) their number of empty endpoints. Let $c(x,y)=\max(0,14-|x-7|-|y-7|)$ and $C(B,p)$ be p's sum of c minus the opponent's. The descriptive formula is

$$E_{raw}(B,p) = \Sigma[r \in R_p(B)]\ w(\text{len}(r),\text{open}(r)) - \Sigma[r \in R_{opp}(B)]\ w(\text{len}(r),\text{open}(r)) + C(B,p).$$

The archived evaluator assigns weights of 1,000,000 to a run of at least five; 50,000 or 5,000 to a four with two or one open ends; 5,000 or 500 to a three; and 200 or 50 to a two. A run of five or more receives the same weight even with closed ends.

Other cases receive zero [21]. These manually chosen weights favor longer, open formations, while the center term adds a smaller positional preference. They are heuristic parameters, rather than a proof that a named threat always outweighs every combination of shorter runs.

```
// Algorithm 2.5 (modified Rust evaluator in [10]; run weights in [21]) —
EVALUATE(B, p):
    score ← 0
    for each occupied cell s:
        for d in the four axes:
            if preceding cell along d has the same color: continue
            (length, open_ends) ← scan maximal same-color run
            value ← WEIGHT(length, open_ends)
            if B[s] is p: score ← score + value
            else: score ← score - value
        score ← score + center contribution signed by color
    return CLAMP(score, -H, H)
```

After each visited run start, score contains the signed run contributions processed so far and the center contributions of previously visited occupied cells. The preceding-cell guard skips interior stones of the same run. Every maximal run therefore contributes once per axis, even when it contains several stones.

For a fixed direction, every maximal run has exactly one first cell: its predecessor is outside the board or has another value. That cell passes the guard and scans to the first nonmatching cell. Every other stone in the same run has a same-color predecessor and is skipped. Thus no run is omitted or counted twice on that axis. Counting the same stone on different axes is intentional because horizontal, vertical and diagonal formations are distinct contributions. For a run of three X stones, scanning from all three cells without the guard could count suffixes as additional runs; the guard prevents this artificial inflation. Since each cell participates in at most one accepted scan per axis, the apparent loop within a loop still has total $O(N^2)$ scan work.

For a board containing only X at (7,7), no length-two-or-longer run contributes, and the center term gives $E(B,X) = 14$. Adding X at (8,7) creates an open horizontal two worth 200; the center terms are 14 and 13, giving 227. For a run shorter than five, changing an empty endpoint to an occupied endpoint changes its open-end classification and corresponding weight. A board boundary also closes an endpoint.

The same two-X board has $E(B,O) = -227$: the geometry stays fixed while the requested perspective changes. If O is added at (9,7), X's horizontal two has one open end and contributes 50, while the signed center contribution is $14 + 13 - 12 =$

15, giving E(B,X) = 65. These are evaluator examples rather than claims of forced wins. The sign means advantage for the requested player, not the color of the last move. The modified Rust recursion keeps that root perspective fixed at every depth: an opponent turn minimizes the same score instead of negating it. Under this evaluator's symmetric weighting and clamp, exchanging the requested player negates the nonterminal score.

Scanning maximal runs in four directions takes $O(N^2)$ time and O(1) auxiliary space beyond the board: each occupied cell belongs to at most one maximal run per direction. This evaluator counts contiguous runs. A split formation such as XX_XX is represented as separate runs at the horizon, although candidate assessment can identify the empty gap as a winning completion. Thus horizon evaluation and tactical move assessment capture different properties of the position.

Illustrative run	Archived Rust weight
.XXXXX.	1,000,000
.XXXX.	50,000
0XXXX.	5,000
.XXX.	5,000
0XXX.	500
.XX.	200
0XX.	50

Figure 2.5. Contiguous-run weights of the archived Rust evaluator
Source: Illustration of the run weights in the archived Rust evaluator [21].

2.5.2 Terminal states and the score hierarchy

Terminal outcomes are checked before the depth cutoff. In the modified Rust snapshot, M = 100,000,000 and q is the ply distance from the root, capped at 225. A root win returns M − q, a root loss returns −M + q, and a draw returns zero. Nonterminal evaluations are clamped to the closed interval [−H,H], where H = M − 225 − 1 = 99,999,774 [10].

```
// Algorithm 2.6 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— TERMINAL_UTILITY(winner, full, root, ply):
    q ← min(ply, 225)
    if winner = root: return M - q
    if winner is the opponent: return -M + q
    if full: return 0
    return NONE
```

The clamp ensures every nonterminal heuristic lies between −H and H. Since M−225 = 99,999,775 > H, even the slowest represented win exceeds a nonterminal score; similarly a loss is below every nonterminal score. A win at q=2 receives 99,999,998 and a win at q=4 receives 99,999,996, so maximizing prefers the earlier win. Conversely, if a loss is unavoidable, increasing q makes its negative score less severe and thus delays the loss. This preference follows from the formula; it is conditional on the search actually reaching and correctly identifying those terminal outcomes. The winner must be decided before a full-board draw when the last empty cell completes a line.

A terminal utility and a pattern weight answer different questions. The run weight of 1,000,000 contributes to a static board estimate; it is not the utility assigned to a proven terminal win. The terminal check precedes evaluation and uses M instead. For an unavoidable root loss at ply two, −99,999,998 is worse for MAX than −99,999,996 at ply four; MIN, representing the opponent's interests, prefers the earlier loss for the root. A zero heuristic indicates a numerical balance under the chosen features, whereas a terminal zero denotes a draw. An empty-candidate zero has a third, implementation-specific meaning described in Section 2.8. These meanings cannot be recovered from the scalar alone.

2.6 Candidate generation and a soft tactical cap

2.6.1 Spatial neighborhood

Considering every empty cell creates a large branching factor. A candidate policy can instead choose empty cells close to existing stones. For cells a and b, Chebyshev distance is d∞(a,b)=max(|a_x−b_x|, |a_y−b_y|). With radius r, a spatial candidate is an empty cell within distance r of at least one occupied cell. The empty board has a special center choice, and a full board has no move. The exact selection, fallback and ranking below describe the archived modified-engine policy, not the legacy bot's different forcing-move generator [10].

```
// Algorithm 2.7 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— RAW_CANDIDATES(B, r):
    if B is full: return []
    if B has no stones: return [center]
    seen ← N × N false array; C ← []
    for each occupied cell s in B:
        for each q in the radius-r square around s clipped to the board:
            if B[q] is empty and not seen[q]:
                seen[q] ← true
                append q to C
    return C
```

For a stone at (7,7), radius one gives a 3 × 3 square with eight empty candidates; radius two gives a 5 × 5 square with 24. With the single stone at (0,0), the radius-two square is clipped to a 3 × 3 corner and yields eight candidates. Multiple stones may nominate the same cell, but seen prevents duplicates. If a radius produces no raw candidates on a nonfull board, the documented selection code falls back to legal moves, avoiding an occupied placeholder. An empty player value or a full board produces no candidates [10].

Each appended coordinate is in bounds, empty and previously unseen. Conversely, every empty cell in a visited neighborhood is appended on its first encounter. This establishes membership in the radius policy. With m occupied cells, board scanning and neighborhood enumeration take $O(N^2 + m(2r + 1)^2)$ time before assessment; the seen matrix and move list use $O(N^2)$ space. The policy reduces the set searched, but can exclude legal moves outside the chosen neighborhood.

With a single central stone, radius two offers 24 candidates instead of 224 legal empty cells. In a hypothetical uniform depth-d tree, reducing width from b to k changes the leaf model from b^d to k^d; this explains why selection can affect search cost more strongly than a small constant-time optimization. Actual widths vary, and later tactical preservation may enlarge them. A radius includes nearby diagonal and orthogonal cells equally under Chebyshev distance. A larger radius admits more distant plans but increases assessment and search work; a smaller radius saves work by making a stronger assumption about where useful moves occur. The raw set is complete for its spatial predicate, not for all strategically useful legal moves.

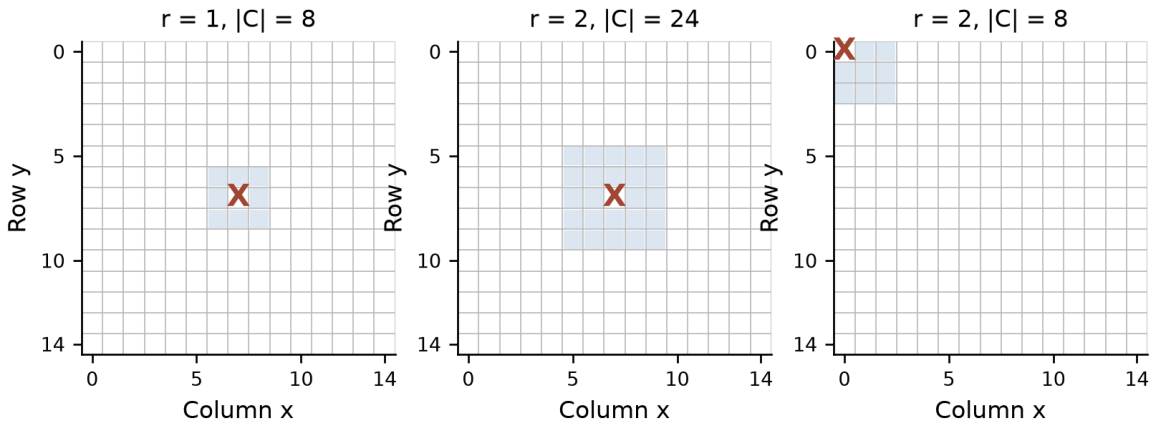


Figure 2.6. Radius and board edges affect candidate membership
Source: research/current_2026_10_23/SOURCE_EXCERPTS.md [10].

2.6.2 Tactical preservation and membership

For each raw move, the documented select_candidates assesses whether it wins for the moving side, blocks an immediate opponent win, and assigns a quick ranking score. Sorting prioritizes a win, then a block, then descending score, with row-major coordinates used to break ties. If a cap is configured, the ranked list is truncated at max(cap,1,tactical_count), so every assessed immediate winning or blocking coordinate remains even if the number of such coordinates exceeds the cap. After selecting membership, disabling move ordering sorts the retained set by coordinate rather than changing its members [10].

The assessment can be implemented without changing the board. At an empty coordinate m, count consecutive own stones on both sides of each axis; the hypothetical stone supplies the missing one, so a sum of at least four marks an immediate win. Repeat the same count for the opponent to mark a blocking coordinate. For X X _ X X, the gap has two X stones on each side: 1+2+2=5, although the static contiguous-run evaluator sees two separate runs before placement. The quick ranking score is a separate quantity: the source assigns own counts 1,2,3,≥4 the values 10,100,1,000,10,000, and opponent counts the values 5,80,900,9,000, sums these over the axes, and adds a small center term. The Boolean win/block keys precede this numeric score in the sort; even a large ordinary ranking score therefore cannot push an assessed immediate tactic behind a non-tactical move [10].

```
// Algorithm 2.8 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— SELECT_CANDIDATES(B, p, policy):
    if p is empty or B is full: return empty set
    raw ← RAW_CANDIDATES(B, policy.radius)
    if raw is empty and B is not full: raw ← LEGAL_MOVES(B)
    ranked ← ASSESS each move for win, block, and quick score
    sort ranked by win first, block second, score descending, coordinate
    tactical_count ← count of entries marked win OR block
    if policy.cap exists:
        retain first max(policy.cap, 1, tactical_count) entries
    if ordering is disabled: sort retained entries by coordinate
    return retained entries and assessment metadata
```

Let C be the raw set, T its marked tactical subset, and c the configured cap normalized to at least one. The selected set is the union of T and enough highest-ranked non-tactical members to reach c when possible. If |T|=2 and c=1, both tactical moves remain, so this is a soft cap, not an absolute maximum. The proof depends on tactical entries being ordered before every non-tactical entry and on every relevant move first being in raw. It does not show that all useful multi-ply tactical moves are preserved, or that a move excluded by the spatial policy is later recovered by the cap.

For k raw candidates, bounded directional assessment takes O(kK), sorting takes O(k log k), and move storage takes O(k), in addition to neighborhood generation. Let C be the raw set, T its assessed win-or-block subset, and c the normalized cap. Selection retains T plus the highest-ranked non-tactical moves needed to reach c, when available. Its total size is min(|C|, max(c, |T|)); it can exceed c. The separate native reference experiment in cap_loss.json illustrates the danger of a hard cap: cap 6 retained six of twenty candidates and omitted both winning endpoints. That result concerns the reference policy, while the modified policy preserves every assessed member of T [22].

2.7 Move ordering versus move selection

A search traverses the retained moves in an order. Ordering permutes an already chosen set; filtering or capping changes its membership and therefore the tree itself. In a complete Minimax evaluation of the same deterministic tree, any permutation produces the same value, although a tied best move can differ if the first best move is retained. In Alpha-Beta search, ordering influences how soon useful bounds are discovered and can alter the number of cutoffs, while preserving the root value under the algorithm's assumptions [23].

```
// Algorithm 2.9 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— ORDER(retained_moves, rank):
    calculate a rank key for each retained move
    sort by the chosen key with a deterministic tie breaker
    return the same moves in the new order
```

If A, B and C have quick scores 10, 50 and 30, score order is B, C, A, whereas coordinate order may be A, B, C. Ranking B first does not establish that it is the best searched move. Minimax still evaluates every retained child; Alpha-Beta may reach a useful cutoff sooner. Sorting k moves takes O(k log k), plus key computation and O(k) move storage in the described implementation. Whether the saved search work outweighs ranking cost is measured in Chapter 5.

The pruning effect comes from bound discovery. In the depth-two example used below, A has MIN leaves [3,5] and B has [2,9]. Searching A first establishes root alpha=3; seeing B's leaf 2 then cuts off its leaf 9, for three evaluated leaves. Searching B first with leaf 9 before 2 requires both leaves because no useful root lower bound exists yet. It then establishes alpha=2, and both leaves under A must be checked, for four evaluated leaves. Both traversals return 3. A ranking heuristic is valuable when it tends to expose such decisive values early, but it may cost more than the work saved, and a poor rank can provide no pruning benefit. This trace concerns only order; removing a leaf would define a different tree.

The documented candidate selection uses fast local information for ranking; the historical reference experiment uses its own evaluation and ordering behavior [10], [13]. Comparisons should hold the board, depth, scoring, candidate membership and terminal rules fixed, changing only ordering. Equal values but different tied moves can both be consistent. A comparison that changes both cap and ordering cannot isolate ordering's effect.

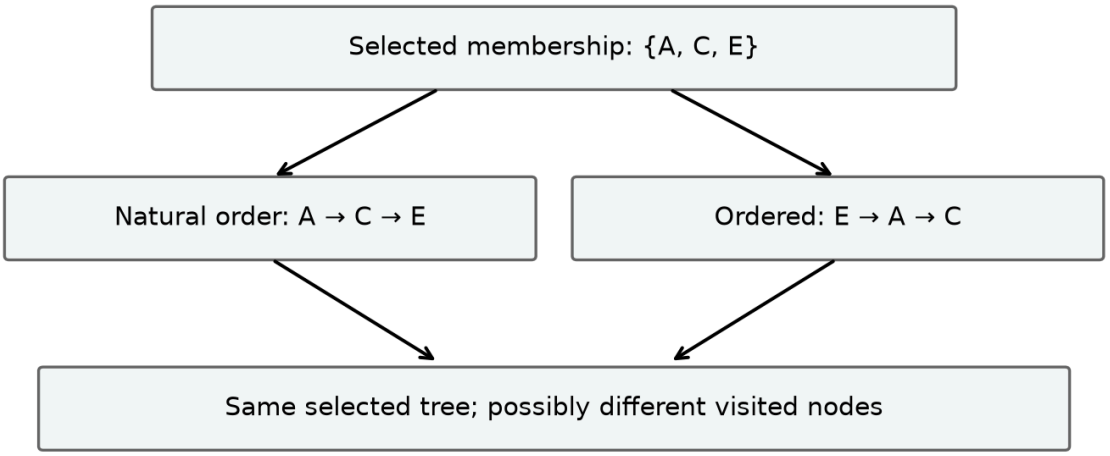


Figure 2.7. Alternative traversal orders on identical membership
Source: research/current_2026_10_23/SOURCE_EXCERPTS.md [10].

2.8 Minimax search

2.8.1 Recurrence and contract

Minimax assumes two alternating decision-makers. MAX chooses a move to maximize a score from the root player's perspective; MIN chooses to minimize it. Its input is the current board, the side to move, a nonnegative depth d in plies, the last move when available, a candidate policy and an evaluator. The output is a value and, at the root, a selected move, or no move for an already finished game. State includes the mutable working board, remaining depth, root perspective and recursion stack. Search must begin from a board compatible with the chosen rules, or validate it separately.

Let U(s,R,q) be terminal utility at root distance q, and E(s,R) the nonterminal horizon evaluation. Transition T(s,a) places the side-to-move stone and changes the turn. For a fixed candidate function A and rule/evaluation configuration, the depth-limited value is

$V(s,d,q) = U(s,R,q)$ if s is terminal;

$V(s,0,q) = E(s,R)$ if s is not terminal;

$V(s,d,q) = \max[a \in A(s)]\ V(T(s,a),d-1,q+1)$ if MAX moves;

$V(s,d,q) = \min[a \in A(s)]\ V(T(s,a),d-1,q+1)$ if MIN moves.

An empty generated set for a nonterminal board must have a specified behavior. The documented value routine returns zero for an empty list, while candidate

fallback is intended to avoid this on a nonfull valid position. That fallback convention should not be silently reinterpreted as a win or as a comprehensive validation of arbitrary positions [10].

At the root, SearchResult carries best_move, score and statistics; it does not carry a distinct invalid-candidate-generation status. In the documented valid-input path, a nonfull board has a center candidate, a neighborhood candidate, or the legal-move fallback, so a positive-depth search is intended to have a move. Depth zero and an already terminal root legitimately return no move for different reasons. Consequently, a caller must interpret the score together with the board state, requested depth and optional move. Introducing a separate error status for an unexpected nonterminal empty set would improve the contract, but would be an implementation change beyond the recorded algorithm [10].

```
// Algorithm 2.10
(research/current_2026_10_23/SOURCE_EXCERPTS.md) —
MINIMAX(B, player, root, depth, last, ply):
   t ← TERMINAL_UTILITY(winner from WIN_AT(B,last), IS_FULL(B),
root, ply)
   if t exists: return t
   if depth = 0: return EVALUATE(B,root)
   moves ← SELECT_CANDIDATES(B,player)
   if moves is empty: return 0   // documented convention for this case
   best ← -infinity if player = root else +infinity
   for m in moves:
      require APPLY(B,m,player)
      try: v ← MINIMAX(B,opp(player),root,depth-1,m,ply+1)
      finally: require UNDO(B,m)
      if player = root: best ← max(best,v)
      else: best ← min(best,v)
   return best
```

The root routine validates an existing result, searches each retained root move, stores a best action and restores the board after each branch. Recursive checks use the just-played coordinate, because a newly formed line passes through that move. The modified Rust implementation uses one recursive kernel whose configuration enables plain Minimax or Alpha-Beta pruning while retaining the same candidate policy and evaluator [10].

2.8.2 Proof, example and complexity

Use induction on d. For terminal nodes or d=0 the procedure returns exactly the corresponding definition. Assume recursive results equal V for depth d−1 and that

each apply/undo restores the parent. At a MAX node the loop's best value after j children is the maximum of the first j child values; at completion it is the maximum over A. The MIN case is symmetric. Each recursive call reduces d, so a finite candidate list guarantees termination. This proves evaluation of the specified finite tree if transition, terminal detection, evaluation and candidate generation satisfy their contracts; it does not prove the evaluator predicts the outcome of an unrestricted game.

In an abstract depth-two tree, MAX has actions A and B; MIN's leaves under A have values 3 and 5, and under B have values 2 and 9. MIN returns 3 for A and 2 for B; MAX chooses A with 3. This two-ply trace illustrates how alternating MAX and MIN backups propagate exact horizon utility values to the root decision.

For branching factor at most $b > 1$ and depth d, the complete generated tree has $O(b^d)$ nodes. Branching varies with position, candidate policy and terminal events. Depth-first search stores O(d) stack frames, up to O(db) live move entries, and an $O(N^2)$ working board. Candidate generation and horizon evaluation add per-node costs, so $O(b^d)$ describes the node growth rather than a complete elapsed-time model.

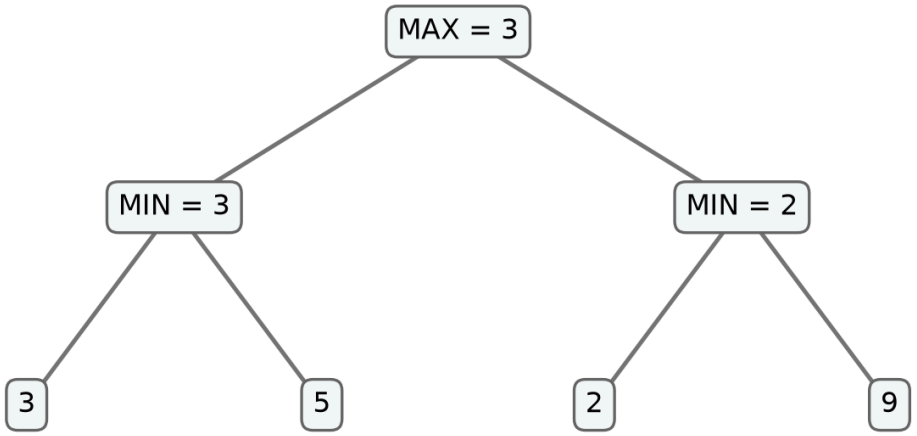


Figure 2.8. A two-level Minimax example

Source: Depth-limited Minimax formulation [16].

2.9 Alpha-Beta pruning

2.9.1 Bounds and pseudocode

Alpha-Beta pruning computes the same root value as Minimax on the same finite deterministic tree under a consistent utility and candidate policy, while sometimes skipping subtrees. alpha is the best lower bound MAX can already secure in the current ancestor context; beta is the best upper bound MIN can already secure. At a MAX node, update alpha after observing a child; at a MIN node, update beta. When alpha ≥ beta, the remaining siblings cannot improve the relevant ancestor's decision. Knuth and Moore analyze Alpha-Beta and the effects of move ordering under stated assumptions [23].

```
// Algorithm 2.11 (modified Rust search kernel in [10]) — ALPHABETA(B, player, root, depth, last, ply, alpha, beta):
    t ← TERMINAL_UTILITY(winner from WIN_AT(B,last), IS_FULL(B), root, ply)
    if t exists: return t
    if depth = 0: return EVALUATE(B,root)
    moves ← SELECT_CANDIDATES(B,player)
    if moves is empty: return 0
    best ← -infinity if player = root else +infinity
    for m in moves:
        require APPLY(B,m,player)
        try: v ← ALPHABETA(B,opp(player),root,depth-1,m,ply+1,alpha,beta)
        finally: require UNDO(B,m)
        if player = root:
            best ← max(best,v); alpha ← max(alpha,best)
        else:
            best ← min(best,v); beta ← min(beta,best)
        if alpha ≥ beta: break
    return best
```

Here restoration occurs before a cutoff is tested. The caller starts with an unbounded window when it needs a root result. A value returned from an internally cut-off node may represent a bound rather than an exact independent value. This distinction becomes essential when such results are cached. The code excerpt increments a cutoff counter at a break; that event count is not the total number of theoretical descendant nodes omitted [10].

This Rust kernel uses a fail-soft return: it returns the best observed child value even when that value lies outside the incoming window. For a MAX node with window [3,8], observing 9 is sufficient to cut off and return 9, which proves a lower bound, not an exact value for the unvisited subtree. Dually, a MIN node can return 2 as an

upper bound below alpha=3. A fail-hard variant would instead return the relevant window boundary, 8 or 3. Either convention can support correct pruning when callers and caches interpret it consistently. Section 2.13's legacy quiescence uses clipped nonterminal returns; it must not be assumed to share the Rust kernel's return convention [10], [15].

2.9.2 Worked cutoff and correctness boundary

In the preceding depth-two tree, searching A first returns 3 and raises the root alpha to 3. At B, MIN encounters 2 first and lowers beta to 2. Since beta ≤ alpha, MIN already has an option no greater than 2, so B cannot improve MAX's available value of 3. The remaining leaf, illustrated as 9, can be skipped. The root still selects A with value 3. Another traversal order may produce fewer cutoffs while retaining the same root value.

Alpha and beta summarize bounds supplied by previously examined alternatives and ancestors. When they cross, the remaining siblings cannot improve the relevant ancestor's choice. Combined with the Minimax and restoration invariants, this preserves the root value on the same finite tree [23]. The archived modified-engine corpus records 408 Minimax/Alpha-Beta pairs, and the separate native reference corpus records 112 pairs with equal root scores. These totals include repeated configurations; they are finite experiment results, presented with their node counts in Chapter 5 [13], [19], [24].

The two cutoff directions are dual. At a MIN node, one examined child with value at most alpha gives MIN an option that is already no better for MAX than an available ancestor alternative; further children can only leave that minimum unchanged or lower it. At a MAX node, one examined child at least beta gives MAX an option already no better for the relevant MIN ancestor than its alternative; further children can only leave that maximum unchanged or raise it. Thus the skipped descendants cannot change the ancestor decision that matters. The argument depends on returning bounds in the correct direction, restoring the board, and using the same retained successors and score semantics as the comparison Minimax tree.

The worst case remains $O(b^d)$ nodes. For a uniform tree with perfect ordering, the ideal minimum leaf count is $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$; for even depth this has the familiar $O(b^{(d/2)})$ growth [23]. Neither bound is an elapsed-time prediction, because node costs and branching vary. Memory remains depth-first: O(d) stack frames and up to

O(db) live move entries, in addition to the board. Equality cutoffs preserve the root value, although tie selection and visited-node counts can depend on traversal order.

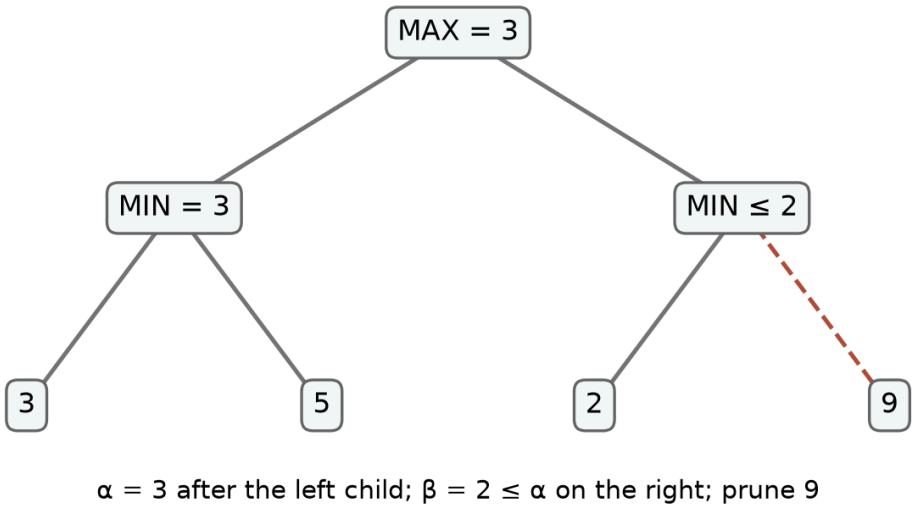


Figure 2.9. Alpha-Beta cutoff on an unchanged tree
Source: Alpha-Beta pruning cutoff formulation [23].

2.10 Iterative deepening and budgets

Iterative deepening performs complete searches at depths 1, 2, … until reaching a configured maximum or a budget interrupts an iteration. Its input is a root position, a maximum depth and a budget. Its output should distinguish the best result from the most recently completed iteration, the completed depth, and the reason for stopping. The working board, search counters, and any table or ordering hints must obey the respective implementation's lifecycle. An incomplete iteration's partially explored best move is not the same as a completed Minimax result for that depth.

```
// Algorithm 2.12 (completed-iteration scheme illustrated by [15]) —
ITERATIVE(B, maximum_depth, budget):
    last_complete ← NONE
    for depth from 1 to maximum_depth:
        if budget is exhausted: break
        result ← SEARCH_AT_DEPTH(B, depth, budget)
        if result was interrupted: break
        last_complete ← result
    return last_complete, completed depth, stop reason
```

The invariant after iteration d is that last_complete corresponds to a whole completed search at d under that iteration's search contract. It follows inductively because the result is replaced only after the iteration returns normally. A separate

abort-safe restoration guarantee is required: interruption in a child must unwind all applied moves. If depth one returns A, depth two returns B and depth three aborts, the completed result is B at depth two. If no iteration completes, a caller must explicitly handle the absence of a completed result; a fallback candidate is useful but should be labeled as such.

Repeated shallow searches consume work, although the deepest full iteration can dominate for a sufficiently large, regular branching factor. Each iteration has the complexity of its selected search; total time is the sum over attempted depths plus ordering and bookkeeping. No hard real-time guarantee follows from this scheme. A search interrupted only at periodic checks can exceed its nominal wall-clock deadline before the next check, especially if individual operations are expensive.

The legacy TypeScript findBestMove uses a performance.now() deadline, checks it at the first search node and every 32 nodes, and catches a dedicated abort sentinel. It initializes a fallback candidate before an iteration completes and reports depth zero if the first iteration aborts. Recursive search uses try/finally to undo moves during unwinding [15]. The historical Rust reference instead uses a node budget, and its aggregate excludes work inside an aborted iteration [13]. The modified Rust engine uses the fixed tier depths shown in Table 2.1 [10]. These are separate budget and counter conventions.

The legacy fallback is the first generated move, with score initialized to zero and completed depth zero; it is not a completed depth-zero search result. Its deadline is start + timeMs − 5. Candidate generation and root ranking run before some of the periodic checks, so the five-millisecond margin does not impose a hard completion bound. Only the dedicated sentinel sets timedOut; stopping between completed iterations because the deadline has passed can still produce reason="completed". Likewise, the no-move label tests null, while an empty root loop can replace the initial null with the internal −1 sentinel. These are limitations of the legacy output contract, not evidence that the active fixed-depth Rust path uses that contract. Algorithm 2.12 expresses the completed-iteration principle; it does not certify every status field in findBestMove [15].

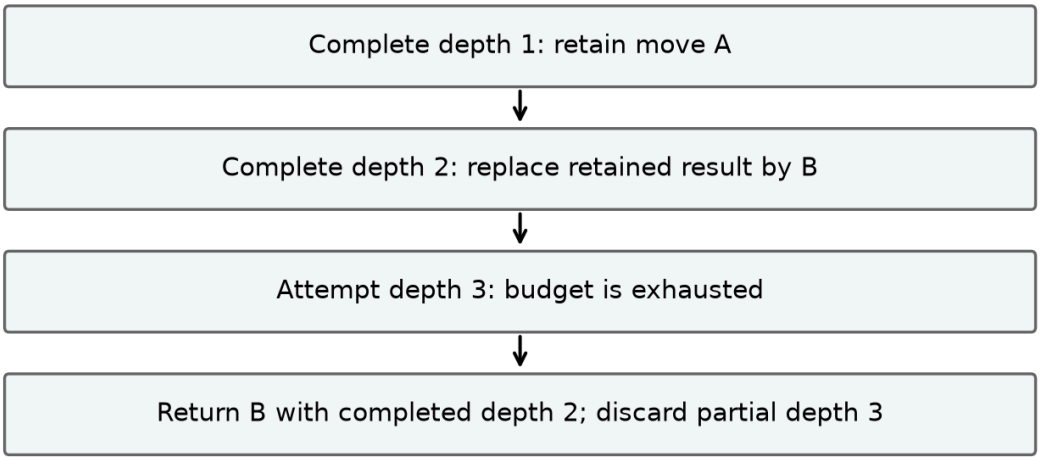


Figure 2.10. Iterative deepening and incomplete work

Source: Illustration of completed-depth handling in the legacy TypeScript bot [15].

2.11 Zobrist hashing and incremental state

2.11.1 Definition and update

A transposition can arise when different sequences of moves lead to the same board and relevant side to move. Comparing all cells at each cache access costs $O(N^2)$, so a compact hash can serve as an index. Zobrist's method assigns a pseudorandom word to each (cell, player) pair and combines the keys for occupied cells with exclusive OR (XOR) [25]. One also needs a side-to-move key if the same occupancy can be evaluated with different players to move. The hash is a compact fingerprint, not mathematical proof that two states are identical.

Define Z[x,y,p] as a fixed random word for a stone, and S as a fixed side key. Under one convention,

H(B,p) = XOR Z[x,y,color] over occupied cells, XOR S if p is the designated second side.

All keys must remain fixed for the lifetime of the table. Applying a move of p at m adds its key with XOR. If the transition changes the side, XOR S as well. Undo repeats the same XOR operations in reverse. The essential identity is (h XOR z) XOR z = h.

```
// Algorithm 2.13 (Zobrist scheme illustrated by the legacy board in [26]) —
FULL_HASH(B, player):
    h ← 0
    for each occupied cell m with stone color c: h ← h XOR Z[m,c]
    if player is designated second side: h ← h XOR S
    return h

UPDATE_HASH(h, m, p, side_changed):
    h ← h XOR Z[m,p]
    if side_changed: h ← h XOR S
    return h
```

For a four-bit example, let h = 1010 and the inserted stone key be 0110, temporarily omitting the side key. Applying XOR gives 1100; applying XOR with 0110 again restores 1010. The same identity applies to each half of the legacy bot's 64-bit fingerprint. XORing both the stone key and side key on make and undo restores the hash in O(1) time; restoring cached line counts has its own cost.

The hash invariant is incremental_hash = FULL_HASH(current board, current side). It holds initially if computed from the board; XORing the stone's key exactly accounts for adding that feature, and XORing the side key exactly accounts for a side change. Repeating those operations removes the features on undo. This proof assumes the same keys, actual transition semantics, and no omitted state features relevant to the hash contract. It does not eliminate collisions.

The legacy board stores the fingerprint as two 32-bit halves and uses a seeded table of stone and side keys. Make and undo also update pattern counts and stoneCount [26]. These fields must all return to their preceding values; the hash alone is not a complete restoration check. In the separate native reference, a winning move can leave to_move unchanged, so the side-key update follows that implementation's actual turn transition [13].

A full recomputation scans $O(N^2)$ cells; one incremental XOR update uses O(1) time. The key table has $O(N^2)$ entries for a fixed number of players; the board and any pattern caches have their own space costs. A finite hash collision remains possible even when incremental and recomputed hashes agree on every tested position. A table may verify a full key or full state, or otherwise accept a defined collision risk. A player key alone also does not encode rules, search depth, evaluator, root perspective or a candidate policy.

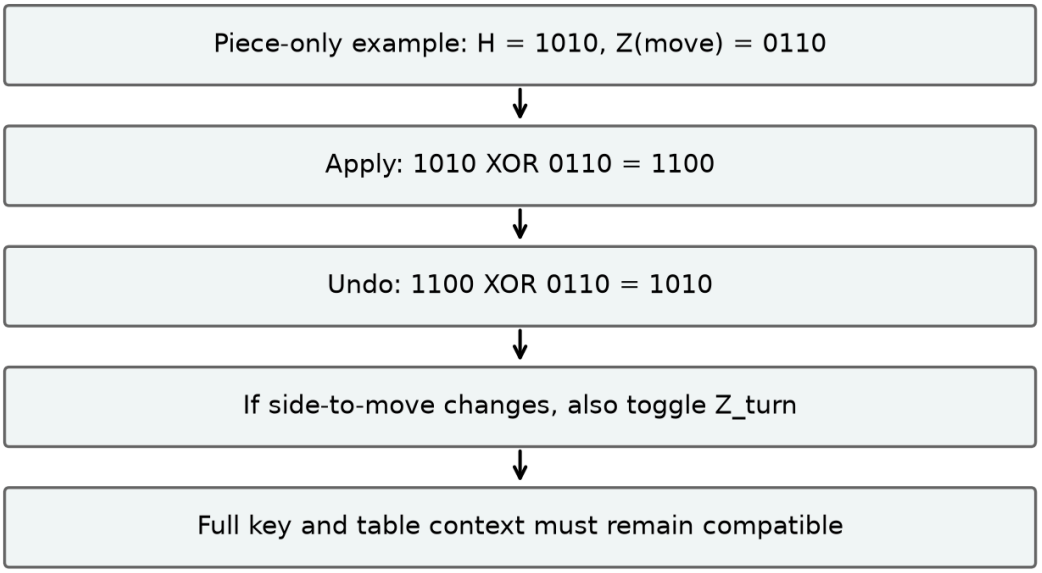


Figure 2.11. Incremental XOR and turn-key handling

Source: CaroGame/src/game/bot/board.ts [26].

2.12 Transposition table: entries, bounds and context

2.12.1 Meaning of stored values

A transposition table (TT) maps a position fingerprint to a previously computed search result. A typical entry stores a hash or verification key, remaining search depth, value, bound type and a suggested best move. The move may help order later search even if the stored score is unusable. A cache lookup is sound only when its represented search problem is compatible with the current one, its depth is sufficient under the selected contract, and its bound justifies the requested cutoff or exact result.

For an original search window $[\alpha_0,\beta_0]$, a common classification is: result $v \le \alpha_0$ is an upper bound; $v \ge \beta_0$ is a lower bound; and $\alpha_0 < v < \beta_0$ is exact, subject to the search's assumptions. Equality is treated conservatively at the boundary. For instance, with window [3,8], a returned 2 is an upper bound, 5 is exact and 9 is a lower bound. Consequently, a stored LOWER bound triggers an immediate cutoff whenever entry.score $\ge \beta$, and an UPPER bound triggers a cutoff whenever entry.score $\le \alpha$.

```
// Algorithm 2.14 (general TT contract; legacy implementation in [15]) —
PROBE(key, required_depth, alpha, beta, context):
    entry ← table.lookup(key)
    if entry missing or key/context is incompatible: return MISS
    hint ← entry.best_move if it is legal in this position else NONE
    if entry.depth < required_depth: return HINT_ONLY(hint)
    if entry.bound = EXACT: return VALUE(entry.score, hint)
    if entry.bound = LOWER and entry.score ≥ beta:
        return VALUE(entry.score, hint)
    if entry.bound = UPPER and entry.score ≤ alpha:
        return VALUE(entry.score, hint)
    return HINT_ONLY(hint)

STORE(key, depth, score, alpha_original, beta_original, best_move):
    if score ≤ alpha_original: bound ← UPPER
    else if score ≥ beta_original: bound ← LOWER
    else: bound ← EXACT
    store key, depth, score, bound, best_move according to replacement policy
```

The probe is sound only when the key, search context, score convention and depth interpretation are compatible. Checking depth, bound type and move legality alone does not establish all of these conditions. Classification uses the original search window, and storage and retrieval must agree about fail-hard or fail-soft returns. A conventional entry.depth ≥ required_depth test accepts deeper results; strict equality with a particular depth-limited horizon requires an additional depth-reuse contract.

The legacy alphaBeta stores best, saves originalAlpha before the loop, and leaves beta unchanged; it classifies best against those original bounds. This is a fail-soft convention at that layer. Its probe returns a stored score only for an exact flag or a bound that closes the requested window; it does not tighten alpha or beta when a bound is insufficient. For example, a compatible LOWER entry of 9 cuts off a query with beta=8, but cannot answer a query with beta=12; the move can still be considered as an ordering hint. A cache hit therefore reuses a statement such as value≥9, not necessarily the equality value=9. The context check shown in Algorithm 2.14 states a requirement: the legacy array implementation checks its hash and depth, but does not separately encode every context field listed in that requirement [15].

2.12.2 Why reuse can be wrong

A useful invariant is: whenever a TT score is reused, it is valid for the current position, depth interpretation, perspective, rule set, evaluation and move-generation context, and its bound permits the intended conclusion. An exact compatible entry

may answer a query; a lower or upper bound is usable only in its permitted direction. If any premise is false, the stored number can change the decision. A full hash comparison narrows ordinary index collisions but still does not guarantee that different game states cannot share a fingerprint.

The native reference records a context-reuse failure: after a clustered depth-4 search, same-root warm reuse returns the same score of 0, but after playing a move and changing the root player, a fresh table returns 0 and the reused table returns 19,800 [27]. The stored scores are root-relative, and the key omits that perspective as well as other configuration fields. This is a result from the reference engine. It motivates matching cache lifetime to the search context. Ply-relative mate scores also require normalization or a restricted reuse scope when stored values can be reached at a different root distance.

The legacy TypeScript table uses fixed-size parallel arrays. A slot is indexed from the low hash half, then both halves are compared before its depth, flag and score are considered. The stored move is used as an ordering hint [15]. The native reference instead uses a hash map [13]. These storage choices do not establish compatibility between search contexts. A hint must refer to a legal retained move before it influences move application, and cache lifetime must match the interpretation of its stored scores.

Expected lookup is O(1) for a suitably configured hash table; actual access cost depends on indexing, collisions and replacement. Memory is proportional to allocated slots or entries, together with container overhead. A hash map's initial capacity is not a maximum, and the size of one entry is not the container's total allocation. A TT can save repeated search work and supply move hints, at the cost of memory and the need to preserve compatible score and state semantics.

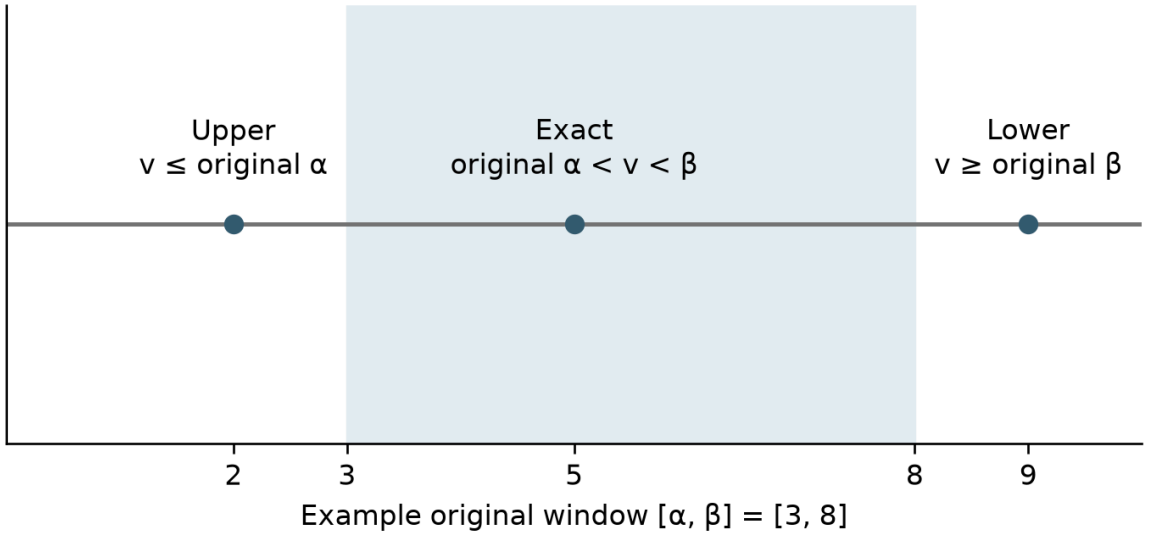


Figure 2.12. Bound semantics in a transposition table

Source: Illustration of bound classes and lookup in the legacy TypeScript search [15].

2.13 Quiescence search in the legacy TypeScript bot

A fixed-depth horizon can fall immediately before a forcing reply, causing a static evaluation to overrate the position. Quiescence search extends selected tactical continuations beyond that horizon. The legacy TypeScript bot implements this extension with a separate depth limit and forcing-move generator [15]. Its result describes that selective continuation tree.

Inputs and state. The legacy routine receives the board, alpha and beta bounds, current ply and a quiescence depth. It works from the perspective of the side to move, uses negamax sign reversal, increments a node counter and checks for abort. Its state includes board cells, side, hash and cached pattern counts. Its output is a bounded score for the selected forcing continuation, or an abort propagated to the caller. It first checks whether either side has a five pattern. It then calculates standPat, applies a beta cutoff if that static score is at least beta, raises alpha if appropriate, and stops at quiescence depth five. Otherwise it generates forcing candidates, orders them and recurses after a move with (-beta,-alpha); each recursive branch uses try/finally to undo [15].

```
// Algorithm 2.15 (CaroGame/src/game/bot/search.ts) — QUIESCENCE(B,
alpha, beta, ply, qDepth):
    count node; check budget or deadline
    if previous player has a five: return -MATE + ply
    if side to move has a five: return MATE - ply
    standPat ← EVALUATE_FROM_SIDE_TO_MOVE(B)
    if standPat ≥ beta: return beta
    alpha ← max(alpha, standPat)
    if qDepth ≥ 5: return alpha
    moves ← GENERATE_FORCING_CANDIDATES(B)
    order moves by their quick score
    for m in moves:
        MAKE(B,m)
        try: v ← -QUIESCENCE(B,-beta,-alpha,ply+1,qDepth+1)
        finally: UNMAKE(B,m)
        if v ≥ beta: return beta
        alpha ← max(alpha,v)
    return alpha
```

The sign change reflects the negamax convention for an evaluation whose perspective changes with the side to move. The procedure returns beta on a fail-high and alpha otherwise, so its return can be a bound rather than an exact unrestricted game value. Its loop invariant is that alpha is at least the stand-pat score and all examined forcing continuations, subject to the window and fail-hard convention; the board must equal the incoming state before every sibling. The depth limit guarantees termination even when forcing candidates continue to exist. It also creates a new horizon.

Negamax combines the two player cases by changing perspective at every edge. If a child position is worth −6 to its side to move, it is worth +6 to the parent player, so the caller uses v=−child_value. Negating the parent interval [alpha,beta] reverses its endpoints: the corresponding child interval is [−beta,−alpha]. For a parent window [3,8], the child receives [−8,−3]. This conversion belongs to both legacy alphaBeta and quiescence; the modified Rust MAX/MIN routine instead passes root-relative values without sign reversal. Mixing these conventions would reverse some preferences twice or not at all. The legacy nonterminal quiescence branches return beta on fail-high and the current alpha otherwise, while its earlier terminal checks directly return mate scores that may lie outside the window. Thus “fail-hard” describes its nonterminal bound handling, not an assertion that every return is clipped [15].

For a simplified numeric example, suppose the window is [−10,10], stand-pat is 2, and the first searched forcing move returns 6 after sign reversal. Alpha rises to 6. If another move returns 12, the routine returns beta, 10, as a fail-high bound. This does not assert that 10 is the exact Minimax value. In a real position, the forcing generator's definition and threat patterns decide which moves are included. The legacy code considers outcomes such as five, open four and four for either side when identifying forcing moves. These categories arise from its own bitmask pattern evaluator, which differs from the snapshot's contiguous-run evaluator [15], [26].

Stand-pat evaluation supplies the initial bound without placing a stone. Because Gomoku requires a move, this is a modeling assumption rather than a legal pass; an immediate cutoff in a forced-defense position need not equal an all-legal-replies result. The forcing set and the five-level extension limit define the selective horizon. With at most $f > 1$ forcing moves per level and q extra levels, a coarse bound is $O(f^q)$ nodes and $O(qf)$ stored move entries, plus per-node generation, ordering and evaluation costs. If $f = 1$, node growth is $O(q)$.

2.14 Legacy bitmask patterns and incremental line counts

The legacy TypeScript bot classifies a line segment with a bitmask: one denotes the player's stone and zero an empty cell, while an opposing stone ends the segment. Its evaluator precomputes categories for segment lengths up to 15 using tests for existing fives and empty cells that would complete a five. The categories are FIVE, OPEN_FOUR, FOUR, OPEN_THREE, BROKEN_THREE and NONE. They follow this classifier's completion-count rules and differ from the modified Rust evaluator's contiguous-run weights [26], [28].

```
// Algorithm 2.16 (legacy pattern evaluator in [28]) —
CLASSIFY_SEGMENT(mask, length):
    if mask contains five consecutive 1 bits: return FIVE
    wins ← number of zero positions which become a five when set
    if wins ≥ 2: return OPEN_FOUR
    if wins = 1: return FOUR
    threats ← number of zero positions which, when set,
              leave at least two immediate winning positions
    if threats ≥ 2: return OPEN_THREE
    if threats = 1: return BROKEN_THREE
    return NONE

COUNT_LINE(B, p, start, step, length):
    split the line wherever an opponent stone occurs
    encode p and empty cells of each segment as a mask
    for each segment of length at least 5:
        increment the returned category bucket only if the category is not
NONE
    return counts
```

For the five-cell segment XX_XX, setting the middle zero creates five consecutive bits, giving one winning completion and the FOUR category. A segment with two distinct winning completion cells is OPEN_FOUR if no five is already present. The category is computed for the whole segment, with the precedence in Algorithm 2.16; an opposing stone or board boundary ends that segment [28].

For a segment of length L, a direct five test examines offsets $j=0,\ldots,L-5$ and checks ((mask >> j) AND 31)=31. To count winning cells, visit each zero bit i, set it in a temporary mask with OR (1 << i), and run the five test. The original mask is unchanged. For XX_XX with bits indexed from the left, mask=11011_2=27; setting bit 2 gives 11111_2=31 and the unique winning cell. For _XXXX_, either endpoint completes a five, so the count is two and the category is OPEN_FOUR. These are completion counts under a specific segment model, not a count of all independent tactical plans: the table returns one category per segment, with FIVE taking precedence over weaker classes.

The board's incremental update subtracts old counts for the four lines crossing a changed cell, makes the move, then adds counts for the new versions of those lines. Undo performs the inverse around clearing the cell. The invariant is that accumulated per-player counts equal a fresh sum of classifications for the board, assuming the untouched lines are unchanged and the four affected lines are correctly identified. Each changed cell lies on exactly one line of each axis, so only

those lines need recomputation. This is a proof sketch for the update scheme, not a proof that the classifier expresses every strategically meaningful formation.

In vector form, let counts[p] be the total category counts for player p, and let affected(m) contain the four board lines through m. The update is:

for p in {X,O}: counts[p] ← counts[p] − Σ[line∈affected(m)] COUNT_LINE(B,p,line)

change the cell at m

for p in {X,O}: counts[p] ← counts[p] + Σ[line∈affected(m)] COUNT_LINE(B,p,line)

Undo repeats the same three steps around clearing the cell; turn, hash and stone count are restored by their own paired updates. Both players must be processed because inserting X also splits or blocks O's segments. For an otherwise empty board with the row fragment O X X _ X X O, the five-cell X segment initially contributes one FOUR. Placing X in the gap subtracts that FOUR and adds one FIVE; the three other crossing lines contain no five-cell threat. Undo subtracts the FIVE and restores the FOUR. If the old line counts were not subtracted, both categories would remain and repeated hypothetical moves would accumulate stale threats. This delta trace illustrates the invariant, while a full recomputation provides an independent consistency check [26], [28].

The legacy evaluatePosition scans board lines and sums the category weights from the requested perspective. Other routines consult the incrementally maintained counts. With fixed-time pattern-table lookup per segment, a full scan over four axes takes $O(N^2)$ time; recomputing the four affected lines takes O(N) per player. The table is initialized for lengths up to 15, so this implementation requires that limit to be reviewed before using a larger board. Its cached counts remain consistent only when every move and undo follows the update procedure [26], [28].

The lookup speed is purchased with initialization work and storage. The allocated Uint8Array has (15+1)×32,768 entries, or 524,288 payload bytes, including unused slots for shorter masks. If the maximum supported length L were allowed to grow, enumerating every mask would be exponential in L. The direct classification routines can perform nested empty-cell trials and five scans, giving a coarse $O(L^3)$ bound per mask and $O(2^L L^3)$ preprocessing overall. Once initialized, lookup is constant time for a supported segment, but constructing its mask still scans its cells. Incremental counts therefore reduce which lines are recomputed; they do not make an arbitrary whole-board evaluation O(1). A source-level extension beyond length

15 must also revise the fixed shift/index layout rather than relying on the larger coordinate capacity of BotBoard.

2.15 Difficulty configurations and the active request path

The application offers local two-player mode and human-versus-bot mode. The latter has six labels: Easy, Normal, Hard, Very Hard, Expert and Master. Table 2.1 combines the frontend request path with the configurations in the modified Rust source and archived difficulty runs [3], [10], [14], [29]. These labels select policies and depths; no player study or strength rating is used to calibrate the spacing between tiers.

Table 2.1. Recorded Rust difficulty configurations and frontend request path

Difficulty	Rust policy	Depth (plies)	Radius / soft cap	Interpretation
Easy	Random	0	All legal empty cells	Bypasses frontend tactics; selects a seeded pseudorandom legal cell without look-ahead.
Normal	Rule-based	1	2 / uncapped	Bypasses frontend tactics; Rust checks wins and blocks, then compares one-ply evaluations.
Hard	Alpha-Beta	2	2 / 15	Frontend tactical check first; otherwise depth-2 Alpha-Beta with nominal cap 15.
Very Hard	Alpha-Beta	3	2 / 12	Frontend tactical check first; otherwise depth-3 Alpha-Beta with nominal cap 12 (API ID 5).
Expert	Alpha-Beta	4	2 / 10	Frontend tactical check first; otherwise depth-4 Alpha-Beta with nominal cap 10.
Master	Alpha-Beta	5	2 / 8	Frontend tactical check first; otherwise depth-5 Alpha-Beta with nominal cap 8.

The tiers change both search method and search width. Easy uses random selection; Normal adds immediate tactical priorities and one-ply evaluation. The remaining tiers use Alpha-Beta at depths 2–5 with radius 2 and nominal caps 15, 12, 10 and 8. Reducing the nominal width offsets some of the growth from increased depth, but these are soft caps: assessed wins and blocks can increase the retained set. For illustration, a uniform tree of width 8 and depth 5 has $8^5 = 32{,}768$ leaves and 37,449 total nodes before pruning. This is a fixed-width model, not an upper bound for the

implemented soft-cap search. The tier timings and actual counters are reported in Chapter 5; configuration alone does not establish a latency bound or a measured progression in playing strength.

AIController receives the application state, round identity and selected difficulty, then assigns a request identifier. Easy and Normal proceed directly to GomokuWasmEngine. The four higher tiers first call the TypeScript tactical resolver for O; a returned move becomes a tactical result without Rust search. Otherwise the controller copies move history into the engine request. This frontend prepass and the Rust Normal rule-based policy are separate decision stages [3], [20].

executeWasmSearch creates a Game, replays the request history through play_move, rejects an unsuccessful replay, calls ai_move, and frees the instance in finally. It converts Rust (x,y) to frontend (row,col). Its elapsed scope, worker-hydrate-and-search, includes reconstruction, search and cleanup, but excludes WASM loading and message delivery [30]. GomokuWasmEngine routes replies by Worker identity and envelope ID [6]. GameController separately checks its turn token, round, pause state, phase, mode and current player before accepting a move. These owners implement resource cleanup and stale-result rejection at different boundaries [2].

2.16 Synthesis and limits of the algorithmic claims

An integrated position connects these mechanisms to the Caro decision problem. Coordinates below are (x,y), meaning column and row. Starting from an empty board, play X(4,7), O(0,0), X(5,7), O(1,0), X(7,7), O(0,1), X(8,7), O(1,1). All eight placements are distinct and in bounds, neither player has won, and X is next. Row 7 contains X X _ X X at columns 4–8; the empty cell (6,7) is X's only immediate winning move.

The gap belongs to the radius-two candidate set because it is within that distance of existing X stones. Directional assessment finds two X stones on each horizontal side, so the hypothetical placement gives $1 + 2 + 2 = 5$ and receives the immediate-win flag. O has no immediate winning completion in this position, making the assessed tactical set $T = \{(6,7)\}$. With a nominal cap of one, the soft-cap rule retains $\max(1, |T|) = 1$ candidate: the winning gap [10].

For a fixed-root Minimax or Alpha-Beta search with root X and positive depth, applying that candidate creates a terminal child at root distance one. Terminal detection precedes the horizon evaluator, so its value is $U = M - 1 = 99{,}999{,}999$

with M = 100,000,000. Both algorithms therefore return the same winning decision on this selected tree. This example demonstrates tactical membership and terminal backup. Because only one root candidate is retained, the trace contains no Alpha-Beta cutoff [10].

The hypothetical placement raises move_count from eight to nine. After evaluating the child, undo clears (6,7) and restores the count to eight, leaving the original decision position available to the caller. These cell-and-count changes take constant time, while candidate generation and assessment retain the costs analyzed earlier. The example connects candidate preservation, terminal backup and state restoration in the same Caro position [10].

The algorithms in this chapter have different responsibilities. Win detection decides the stated terminal rule; board mutation maintains a position; random and rule-based choices provide baselines; the evaluator assigns an approximate horizon value; candidate generation restricts the moves explored; ordering permutes the retained moves; Minimax evaluates the resulting tree; Alpha-Beta skips provably irrelevant branches of that same tree; iterative deepening manages progressive search; and hashing plus a transposition table aim to reuse compatible earlier work. Quiescence extends a selected class of tactical moves at a horizon, but its own assumptions and depth limit remain.

The current decision path uses frontend tactical checks and a Worker-hosted WASM request. The modified Rust source explains board updates, Freestyle detection, candidate generation, horizon evaluation and fixed-depth Alpha-Beta. The legacy bot supplies the iterative-deepening, hashing, transposition-table and quiescence examples. Their contracts are examined independently: candidate selection defines the searched tree; ordering changes its traversal; and Alpha-Beta preserves Minimax's root value for that same tree under the stated assumptions.

Chapter 5 evaluates the recorded implementations using native search experiments, WASM parity checks and browser Worker observations. Each result is interpreted with its own position, configuration, platform and counter definition. The theoretical arguments in this chapter establish properties of specified algorithms; the experiments assess those properties on the recorded finite fixtures [13], [19], [31], [32].

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

3.1 Decision problem and algorithmic requirements

This chapter states the system conditions required for a search result to solve the intended Caro decision problem. The player supplies accepted moves in local two-player or computer-opponent mode; the AI service receives a board snapshot and proposes a move; and host integration supplies lifecycle signals such as pause and resume. GameController validates these actions and owns accepted state transitions. The requirements below therefore define how a legal position becomes an algorithm input, when its returned move may be committed and how obsolete computation is rejected [2], [4], [33].

Table 3.1. Algorithmic requirements AR01–AR09

Requirement	Trigger and input	Required behavior and observable result
AR01 Position and legal-state contract	A board position, side to move, ordered history and a proposed placement are supplied.	Require integer coordinates from 0 to 14, an empty target cell and a state consistent with the selected rules. Accepted moves must keep board cells, move count and history consistent; validation limits must remain explicit.
AR02 Define the generated move space	Search receives a valid position, player and candidate policy.	Distinguish all legal empty cells from the generated candidate set. Generated coordinates must be in bounds, empty and unique. State the empty-board choice and nonterminal fallback rather than interpreting an unexplained empty list as a terminal result.
AR03 Determine terminal outcomes	The root position is examined or a hypothetical move is applied.	Apply the Freestyle rule: a contiguous line of at least five stones wins, including overlines. Check a win before a full-board draw. Use last-move scanning only under its stated predecessor-state assumption.

Requirement	Trigger and input	Required behavior and observable result
AR04 Preserve score semantics	A terminal or depth-limited position requires a numerical value.	Keep the evaluation perspective fixed within the chosen search convention. Distinguish terminal utility from heuristic pattern weights, preserve the terminal-score hierarchy, and interpret zero and mate distance according to their stated contracts.
AR05 Compare Minimax and Alpha-Beta on the same tree	Both algorithms receive the same position, depth, terminal rules, evaluator and generated successors.	Completed searches must return the same root value on that finite tree. Apply Alpha-Beta bounds in the correct direction. Allow different tied best moves when traversal order changes; do not extend selected-tree equivalence to excluded legal moves.
AR06 Preserve assessed immediate tactics	The modified Rust policy ranks raw candidates and applies a nominal cap.	Retain every assessed immediate winning or blocking coordinate even when their count exceeds the nominal cap. State that this soft-cap property depends on raw membership and assessment; it does not establish completeness for deeper tactics.
AR07 Separate ordering from selection	A retained candidate set is traversed with or without move ordering.	Ordering must permute the same retained set. Keep membership fixed when measuring its effect on Alpha-Beta; distinguish recursive visits from the additional work required to rank moves.
AR08 Bound search and describe its output	A fixed-depth search completes, or a separately studied budgeted iteration stops.	Use the stated depth, candidate and extension limits to define the finite search. Report the selected move, score convention, completed depth and available counters with their source and measurement scope. Do not label an interrupted iteration or fallback as a completed deeper result.

Requirement	Trigger and input	Required behavior and observable result
AR09 Restore search state and guard commitment	A hypothetical branch exits or an asynchronous search result reaches the application.	Restore every state field owned by the search before exploring a sibling or returning through a supported exit. Commit a proposed move only if it remains legal for the current turn and round; discard stale results and release obsolete Worker resources.

These requirements separate three questions: whether a position and transition are valid, what value the selected search tree defines, and whether a returned proposal still belongs to the current application state. Board and rule operations establish the first boundary; evaluation, candidate policy and recursion establish the second; controller and Worker lifecycle checks establish the third [2], [10], [18], [30].

The following criteria explain AR01–AR09 and connect them to the algorithms in Chapter 2 and the experiments in Chapter 5. They state the required contracts; the later results and limitations identify the cases actually checked.

AR01–AR03 define the decision problem before search begins. A position must have legal cell values and a consistent history, and a placement must use an integer in-bounds empty coordinate. The pointer path supplies integer indices through Math.floor, and the WASM result adapter checks integer coordinates; Chapter 6 identifies the direct-call validation limit. Candidate generation defines a subset of legal moves, with explicit empty-board and fallback behavior. Terminal evaluation applies the Freestyle rule and checks a win before a full-board draw; last-move detection assumes an unfinished predecessor position [2], [8], [10], [18], [30].

AR04–AR06 constrain the value being computed. Terminal utility and horizon evaluation have different meanings, and the chosen perspective must remain consistent throughout recursion. Minimax and Alpha-Beta are compared only after fixing the position, depth, rules, evaluator and candidate tree. The modified Rust soft cap retains all assessed immediate wins and blocks from its raw set, whereas the separate reference hard-cap example shows how changing membership can remove a winning move. These properties concern the stated policies and do not establish that selective search finds every useful continuation [10], [13], [22].

AR07–AR09 separate computational work from decision validity. Ordering changes traversal while preserving the retained move set, so ranking evaluations and recursive visits must be reported separately. Fixed-depth, quiescence and budgeted searches have their own stopping and completion contracts. Every explored branch must restore the state fields it changes, and an asynchronous result may update the match only while its turn and round remain current. Cancellation and result validation therefore protect the meaning of the algorithm input and output rather than serving only as interface behavior [2], [6], [10], [13], [15], [30].

Search comparisons hold the position, rules, root perspective, depth, evaluation and candidate membership constant. Minimax and Alpha-Beta should then return the same root value; equally valued moves may differ with traversal order. The recorded native corpus contains 408 paired comparisons and 820 root-restoration checks. Separate Node/WASM tests exercise the copied binary, controller tests exercise asynchronous guards, and browser records exercise the Worker and selected UI interactions. Chapter 5 reports these results by environment.

3.2 Non-functional requirements and acceptance boundaries

Correctness means preserving legal state transitions, the agreed rule and the relation between board cells and history. Maintainability means keeping rule evaluation, rendering, input conversion and asynchronous search behind identifiable interfaces. Responsiveness motivates the Worker boundary for substantial search, while correct cancellation requires more than moving work off the main thread. Reproducibility requires a source revision, binary identity, configuration and measurement procedure rather than a screenshot containing a duration.

The Rust/WASM route runs inside a dedicated Web Worker, separating its synchronous search from the main thread that handles React and Pixi. Cancellation terminates the Worker, while controller guards determine whether a completed response still belongs to the active turn. Easy and Normal use Rust baseline policies; the four higher tiers use fixed-depth Alpha-Beta unless the TypeScript tactical resolver supplies an immediate move.

3.3 Architecture and ownership

The system has five principal responsibilities. React owns interface composition and subscribes to a controller snapshot. GameController owns the game-facing action boundary, current-player choice, AI-turn lifecycle and notification. The core owns board cells, legal-move history and rule evaluation. Pixi owns the visual board and converts gestures through an input/coordinate layer. The AI adapter owns

search dispatch and Worker resources; it proposes a move but does not directly mutate the authoritative application board [2], [4], [5], [18].

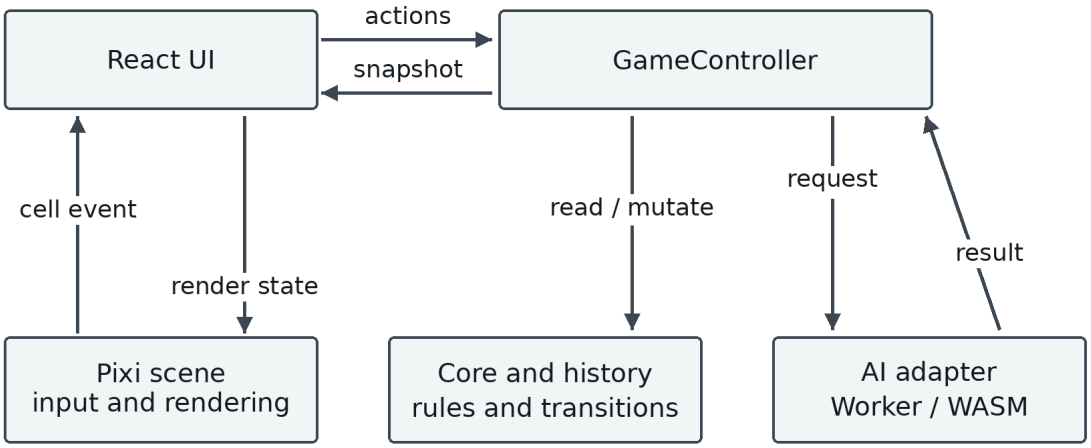


Figure 3.1. Responsibilities in the playable application
Source: CaroGame/src/app/GameController.ts [2].

The scene emits a cell event through the callback supplied by the React application, which calls GameController.playCell. The controller updates domain state and publishes a snapshot. React then passes stones, the last move and winning cells to the scene for rendering. AI requests use copied history, and returned moves re-enter the controller and domain validation path before becoming visible board state.

A snapshot contains the match record, current player, thinking and pause flags, board stones, history, last move, last AI result and error. The controller copies move records and their cell coordinates when creating it. Subscribers read a stable cached snapshot until a notification creates a new one. This matters for React subscriptions: returning a newly constructed object on every read could produce unnecessary updates or unstable identity, whereas mutating a previously delivered object could conceal a change. The observed implementation uses an explicit cached value and listener set [2].

A Web Worker executes independently of the main interface script [34]. GomokuWasmEngine owns the Worker object, transport identifiers and pending promises. GameController owns turn validity. Resource cancellation stops obsolete Worker execution, and the controller token also rejects a response that resolved before cancellation reached it [2], [6].

3.4 Board state, history and legal transition contract

The frontend GameState stores one value per cell and an ordered move history. A move records its player, cell and index; history is also the input used to reconstruct a fresh Rust game for a search. These two uses require consistency. If a board cell changed without a corresponding history update, the display could show one position while the engine searched another. The transition contract therefore relates the occupied-cell set to the history sequence rather than treating them as independent data [18].

The board has $N = 15$ and $N^2 = 225$ cells. Frontend coordinates are (row, column); Rust uses (x, y), where x = column and y = row. A row-major linear index is row × N + column. The pointer transform applies floor before checking board bounds. The WASM result adapter checks integer coordinates and bounds before converting them to a frontend cell [30], [35].

The playMove operation checks bounds and occupancy, commits an accepted move to cells and history, and evaluates the supplied win rule. GameController supplies the current player and checks pause, phase and AI-turn availability. Geometric win detection remains in the rule object, while match and request lifecycle checks remain in the controller [2], [36].

The history sequence is not merely a move counter. It preserves alternation and the exact order needed for hydration. A full board without a winner is declared a draw after the win test. That order matters on the last empty cell: a winning final placement must remain a win rather than being classified as a draw merely because the board is now full. The controller's finishMove uses this ordering [2].

3.5 Match phases and state transitions

MatchController defines setup, ready, playing and game-over. Its initial state is ready, local two-player mode, normal difficulty and round identifier zero. Calling playCell from setup or ready invokes startRound, which increments the round identifier, clears the old outcome and enters playing. An accepted win or full-board draw changes the match to game-over. The thinking flag belongs to the AI request lifecycle; it is not a fifth value in the phase enumeration [2], [33].

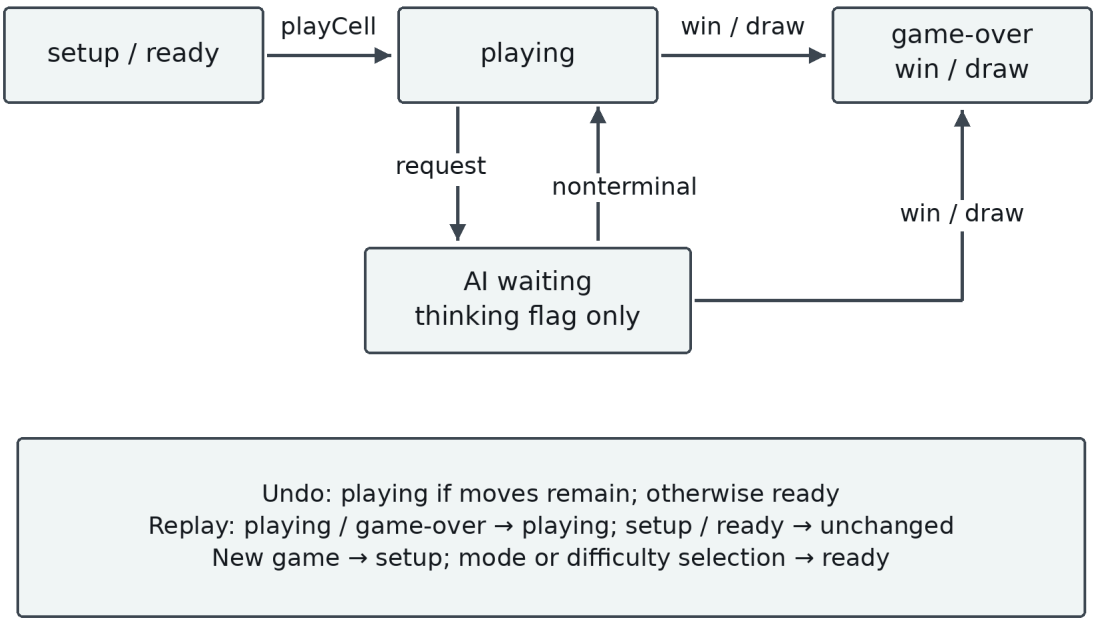


Figure 3.2. Match phases and the separate AI waiting flag
Source: CaroGame/src/game/behaviors/match/MatchController.ts [33].

New Game increments the round identifier and enters setup. Selecting a mode or difficulty clears the previous game and enters ready. Replay enters playing when the previous phase is playing or game-over. Undo calls resumeAfterUndo, increments the round identifier and chooses playing if moves remain, otherwise ready.

Pause is independent of match phase. Pausing preserves the board and phase, blocks input and invalidates the active AI request. On resume, the controller starts a new AI request only if the match is still playing in AI mode and O still owns the turn [2].

A state-machine invariant can now be stated. A result may commit only when its request belongs to the active AI turn and round, the application is unpaused, the phase is playing, the mode is AI and O is the current player. None of these predicates is replaced by the fact that a promise resolved successfully. Success describes the completed computation; the predicates describe whether that computation is still applicable.

3.6 Asynchronous request protocol and cancellation

An AI action captures a new turn token, the round identifier and difficulty before setting thinking state. The request contains a copied move history. The transport wrapper adds its own message identifier so the correct pending promise receives the

corresponding Worker response. The returned domain result carries its request/round identity, difficulty, move and statistics. The message identifier, AI request identifier and round identifier serve related but different scopes; they should not be collapsed into a single undocumented counter [2], [3], [6].

On completion, the controller compares the captured token with its current token, checks pause and the returned round identifier, and repeats the phase, mode and current-player conditions. Only then does it clear thinking state, save feedback and pass the proposed O move through the same domain move behavior used elsewhere. A rejected returned move produces an error instead of being painted directly. A missing move in an active round is also an error. This keeps the authoritative board inside the domain path [2].

A concrete race illustrates the mechanism. Suppose request A is searching round 8. The person invokes New Game before A finishes. The controller increments its token, clears state and changes the round. Even if A's response was already queued, its captured token no longer matches. The promise resolution cannot insert A's move into the new board. Meanwhile, cancelPending rejects pending transport promises and terminates the old Worker, so future requests construct a new one. The token guard and resource cancellation address different parts of the race [2], [6].

A caught error changes feedback only while its captured turn token remains current. GomokuWasmEngine ignores messages from a Worker that is no longer active and ignores envelope identifiers absent from its pending map. Together, these checks keep an obsolete completion or rejection from replacing the current turn’s feedback.

3.7 Coordinate transformation and gesture interpretation

The scene works with a camera translation (cx,cy), a positive zoom z and cell size S = 56. For a screen position expressed in the scene's coordinate system, the inverse transform is worldX = (screenX − cx)/z and worldY = (screenY − cy)/z; the selected column and row are floor(worldX/S) and floor(worldY/S). Conversion back to a cell's world origin multiplies its column and row by S. A center point for drawing a stone is obtained by adding half a cell in the renderer, rather than changing the indexing contract [35].

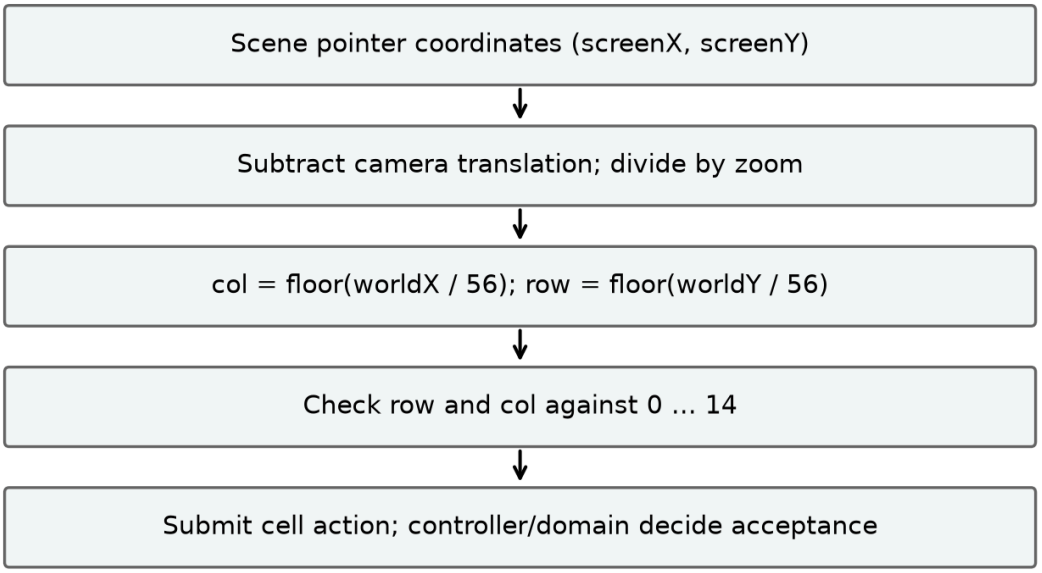


Figure 3.3. Inverse camera transform before board bounds checking
Source: CaroGame/src/board/spatial/CoordinateTransform.ts [35].

The floor operation is important near boundaries. A negative world coordinate must remain out of bounds; rounding it toward a nearby valid cell could turn a margin tap into a move. For example, with camera x = 20, zoom = 0.5 and screen x = 19, the world x is −2 and the column is −1. The subsequent bounds check rejects it. With screen x = 230, world x is 420 and the column is 7. The same calculation on y yields the row. The report uses scene screen coordinates; client-page coordinates must first be converted by the event/rendering system that supplies them.

BoardInputController uses idle, pressed and dragging states, with a drag threshold of six scene screen units. Pointer-down records the current position. Pointer-move measures displacement from the previous event; a displacement greater than the threshold changes pressed to dragging and produces a pan delta. Pointer-up emits a cell tap only while the state is still pressed and the resolved cell is in bounds [37].

CaroScene handles pointer-up outside the stage through the same release handler and suppresses pointer processing while paused. It fits the board using the smaller width and height scale ratios, with reduced padding below a viewport width of 500. Camera translation and scale are applied to the world container, leaving domain coordinates unchanged [5].

3.8 Search-result contract and presentation of telemetry

The Rust result validator requires in-bounds integer x and y coordinates, a finite score, matching difficulty, a recognized strategy and nonnegative safe-integer

counters. The counters report visited nodes, leaf evaluations, cutoffs, ordering evaluations, generated candidates and completed depth. A coordinate-only result or another incompatible payload raises a contract error [14], [30].

The adapter maps the fields into application terminology and explicitly labels the source as Rust. Its elapsed timer starts before construction and history hydration, includes ai_move and free, and ends after cleanup. It excludes module loading and message delivery. A displayed duration therefore has a defined scope: it is not a complete user-perceived round trip and is not a measurement of only the recursive search. The tactical route instead labels its source as tactical, reports zero Rust search counters and a null score. Zero counters on that route mean Rust search was bypassed, not that a nontrivial Rust tree required no work [3], [30].

Configured depth describes the requested policy; completed depth is returned telemetry. Candidate selection, terminal states and the selected strategy determine the work represented by that result. The interface therefore presents depth alongside strategy, source and counters rather than using a difficulty label as a measurement.

3.9 Verification scenarios derived from the design

Table 3.2. Verification scenarios and expected outcomes

Scenario	Input/event sequence	Required observation
S01 Legal placement	Tap an empty in-bounds cell while the current turn is available	One history item and stone added, correct outcome and next player.
S02 Rejected placement	Tap an occupied or out-of-bounds cell	No board/history mutation.
S03 Last-cell win	Place the final empty stone so it completes a winning line	Win is recorded before a possible draw condition.
S04 Delayed AI response	Request AI, change the round, then deliver the old response	Old result cannot commit a move or overwrite current feedback.

Scenario	Input/event sequence	Required observation
S05 Undo before reply	Human move followed by Undo while a request is pending	Request invalidated and board/turn restored consistently.
S06 Undo after reply	Completed human-plus-AI pair followed by Undo	Correct number of plies removed and feedback cleared.
S07 Worker failure	Worker raises an error, followed by a valid retry	Pending requests are rejected; an error is visible; a new Worker can serve a new request.
S08 Pause/resume	Pause during O's AI turn, then resume	Input is suppressed; obsolete work discarded; needed current turn can restart.
S09 Resized scene	Change viewport size after board content exists	Transform and visible board update without changing domain cells.
S10 Telemetry mismatch	Use a result missing counters or with invalid coordinates	Adapter rejects the incompatible result rather than inventing fields.

Scenarios S01–S10 cover legal state changes, stale-response rejection, error recovery, pause/resume, resizing and response validation. The test and browser results in Chapter 5 are organized around these module boundaries and their recorded environments.

CHAPTER 4. IMPLEMENTATION

4.1 Toolchain and project organization

The application declares React 19, React DOM, PixiJS 7.4, TypeScript, Vite, Vitest and i18next. Its scripts provide development serving, production build, preview, formatting, static typechecking and automated tests. The frontend package-lock.json and the Rust Cargo.lock files record resolved dependency versions for their respective packages [38].

The source folders reflect the design boundaries. app contains the game controller. game/core supplies state, bounds, coordinates and rules. game/behaviors implements move and match transitions. game/ai defines the service and tactical route, while game/wasm provides the worker, adapter, typed contract and generated engine package. board/input, board/spatial and board/render separate gestures, transforms and drawing. pixi integrates them into a scene; React components provide controls and feedback around the canvas [1].

The implementation is examined through the decision pipeline: accepted moves establish the board and history, the selected AI route computes a proposal, and validation determines whether that proposal may update the current position. Domain state and rules remain independent of React components and Pixi objects. AI modules consume domain history, while rendering modules receive state-derived drawing data. These boundaries support focused checks of legal moves, coordinate mapping and controller lifecycle; the interface provides an observable setting for the algorithms rather than a substitute for their evaluation [5], [38], [39].

4.2 Accepted move processing

The accepted-move path establishes the position that the search will receive. GameController.playCell rejects paused input, invokes startRound from setup or ready, checks ended or AI-owned turns, and delegates cell validation and placement to playMove. Only an accepted domain move proceeds to outcome evaluation, current-player selection and notification. A nonterminal result with O to move in AI mode then triggers a search request based on the updated history. The ordering of the initial phase transition and validation remains a boundary case discussed in Chapter 6 [2], [36].

The sequence is deliberate. Notification occurs after the accepted domain change, so the interface receives a coherent snapshot. The AI request is based on the updated history, not on a speculative screen state. Rule evaluation occurs through the supplied FreestyleGomokuWinRule. The scene never paints a permanent stone

merely because a pointer event arrived; it renders the state returned through the controller. This permits the same move contract to reject an occupied cell consistently in the UI and in tests.

History reversal reconstructs an earlier decision state. The controller invalidates AI work and clears feedback before removing one ply in local two-player mode, or the applicable human–AI pair in AI mode. If the last recorded player is O it removes two plies; otherwise it removes one. It then derives the current player from the remaining history, updates the match phase through resumeAfterUndo and publishes the restored state. Thus a later search must use the restored position rather than a result computed for the discarded continuation [2].

4.3 AI dispatch, Worker execution and WASM hydration

AIController.requestMove assigns a request identifier and inspects the difficulty. Easy and Normal bypass the TypeScript tactical shortcut so they actually execute the Rust Random and Rule-based policies. Higher tiers ask the tactical resolver for an immediate forced action for O. If one exists, the controller resolves a typed result with tactical source, null score and explicitly zero Rust counters. Otherwise it copies the entire accepted history and delegates to the WASM engine wrapper. This distinction is visible in the result contract and should remain visible in experimental interpretation [3], [20].

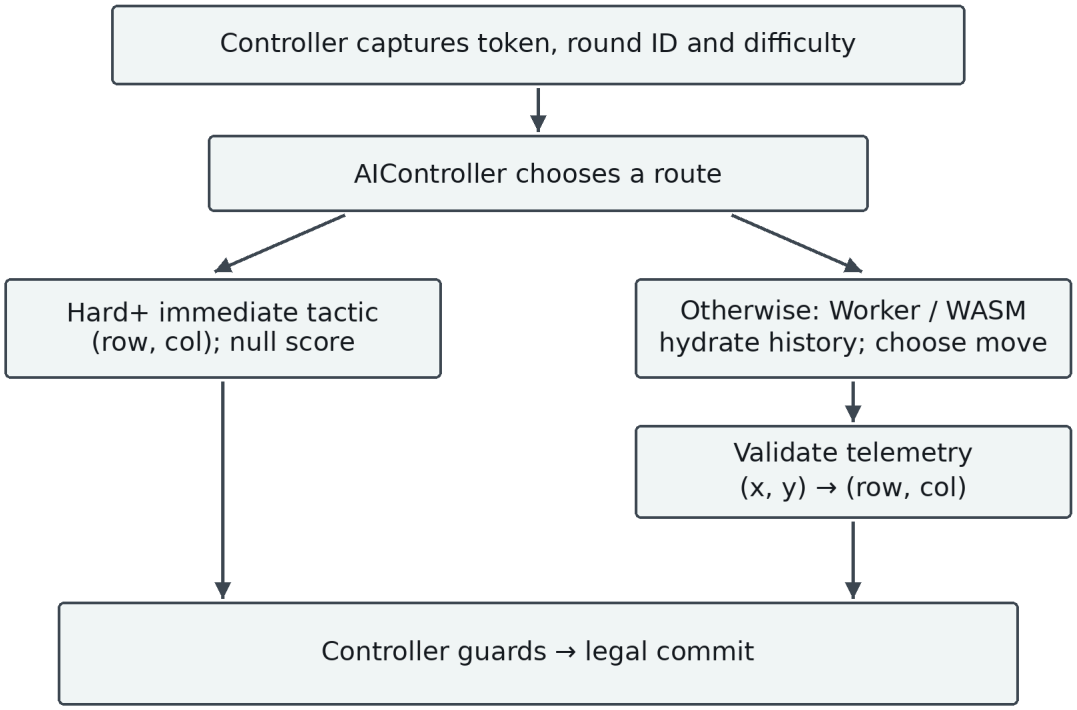


Figure 4.1. Tactical and Rust routes converge at the guarded commit
Source: CaroGame/src/game/ai/AIController.ts [3].

GomokuWasmEngine creates a module Worker lazily, registers a pending promise under a transport identifier and posts a search message. worker.ts initializes the generated module and calls executeWasmSearch. The same adapter is used by the binary tests, so both routes share coordinate and telemetry conversion. A returned envelope resolves its matching promise, while a structured error rejects it [7], [30].

Hydration creates a fresh Rust Game object and replays each history move as (col,row). Each replay call must succeed. If any move cannot be applied, the adapter throws an error identifying the move rather than searching a partially reconstructed board. After hydration, it calls ai_move with the mapped difficulty identifier, validates the returned object and maps (x,y) back to (row,col). A finally block frees the game even if hydration, search or result validation fails. This is an explicit resource lifecycle, not an assumption that JavaScript garbage collection releases every Rust allocation promptly [30].

The generated package contains JavaScript bindings, TypeScript declarations and the WebAssembly binary loaded by the Worker. Selected modified-engine source excerpts describe the recorded Rust search, while the bindings expose its interface to JavaScript. The recorded binary hash identifies the package used for the parity

and browser observations, and Appendix B lists the associated source and build records [10], [40].

4.4 Board rendering and interface synchronization

CaroScene creates a transparent, antialiased Pixi application associated with the supplied canvas. It builds a world container with separate grid, highlight/effect and stone layers. The scene stores the latest render stones, last move and winning-cell set supplied by syncState; these are rendering inputs derived from the controller, not independent move authority. The camera subscription reapplies translation/scale and refreshes the visible board. A ticker callback checks whether the viewport dimensions have changed and refits when necessary [5].

GridRenderer draws board geometry, StoneRenderer maps player/cell records to pieces, and HighlightRenderer draws the last move and winning cells. Input passes through BoardInputController and BoardCoordinateTransform before the scene reports a cell event. This separation allows coordinate and bounds tests to run independently of domain win-rule tests.

The React application subscribes to controller snapshots, wires controls to controller actions and passes board state to CaroGameCanvas. The canvas component owns the scene, resize observer and synchronization effects. On unmount it disconnects the observer and destroys the scene; scene destruction unsubscribes the camera and destroys the Pixi application and children. The controller separately disposes its AI service and Worker resources [4], [5], [6].

The interface includes translation resources for its supported labels and search details. Those resources map keys to displayed text; they do not change difficulty semantics or engine telemetry. An elapsed label, a completed-depth label and a strategy/source label should all refer to the current AI result. Undo, reset or a mode change clears stale feedback in the controller, ensuring that an old score is not displayed as though it belonged to the new board [2], [41].

4.5 Presentation website and interactive explanation

The separate present application explains the algorithms through weekly material, diagrams, benchmark sections, a figure inspector and a browser-side demonstration engine. These components provide instructional examples of search, candidate policies and advanced methods. Their inputs and execution paths remain distinct from the playable application and native experiment harnesses, so a demonstration output must be interpreted under its own model rather than attributed to the authoritative Rust/WASM search [39], [42].

The presentation uses tree animations, diagrams and figures to explain the algorithms. Quantitative examples are associated with the original JSON or CSV datasets described in Chapter 5. The Python snippets retain their own inputs and weights so that each calculation can be read and executed as a self-contained teaching example.

InteractiveCaroBoard uses the presentation application’s own utility engine and state. It lets readers explore move selection within the presentation site. The game application continues to use its separate GameController and Rust/WASM Worker route. Historical component diagrams are retained as development illustrations and are distinguished from the current architecture in Figure 3.1.

4.6 Python/Pyodide execution and four instructional programs

The presentation's Python runner loads or reuses a Pyodide instance. It captures stdout and stderr, loads packages inferred from imports when supported and executes code through runPythonAsync. The returned string is displayed to the reader, while caught errors become visible output. Pyodide executes Python in a browser environment based on WebAssembly; its presence does not mean the game AI is implemented in Python. The game engine's Rust/WASM path and the teaching runner are separate consumers of web execution technology [43], [44], [45].

The runner caches the initialized Pyodide instance and queues callers while initialization is in progress. It redirects stdout and stderr into the output panel, reports package-loading progress and executes the selected code asynchronously. The four supplied programs cover horizontal patterns, weighted feature scoring, candidate geometry and arithmetic on an embedded benchmark summary.

4.6.1 Pattern scanner

The first program constructs a 15 × 15 board using NumPy, with a pure-Python fallback. It places four X stones horizontally and three O stones vertically. detect_patterns scans rows, counts maximal contiguous runs and classifies five, open four, closed four, open three and closed three from their endpoints. For X, the supplied board produces one horizontal open four [44].

The input is an integer board matrix and a selected player; the output is a dictionary of horizontal pattern counts. Advancing from the beginning to the end of each run counts it once. Endpoint checks distinguish open and closed runs without leaving the board. Broken patterns such as XX.XX require a gap-aware detector beyond this contiguous-run example.

4.6.2 Heuristic calculator

The second program calculates a score from manually supplied pattern counts and a demonstration weight dictionary. The instructional attack/defense cases yield 94,000 and −7,000 under those inputs and the defensive multiplier. These are arithmetic outcomes of the supplied feature counts. The application's Rust evaluator scans the board using a different set of weights [21], [44].

The calculation separates attacking features from weighted opponent threats. Its input consists of two feature-count dictionaries, a weight dictionary and a defensive multiplier of 1.2. A negative score means the weighted formula favors the opponent in that example. Forced wins and terminal utility are handled separately in the search models described in Chapter 2.

4.6.3 Chebyshev candidate example

The third program places five stones and compares empty cells with candidates within Chebyshev radius two. It obtains 36 selected cells from 220 empty cells, so the reduction for that constructed board is $(220 - 36)/220 \times 100 = 83.64\%$. The denominator excludes occupied cells and the candidate mask excludes those cells as well. Replacing this per-board calculation with an average from a different experiment would change the meaning of the number [44].

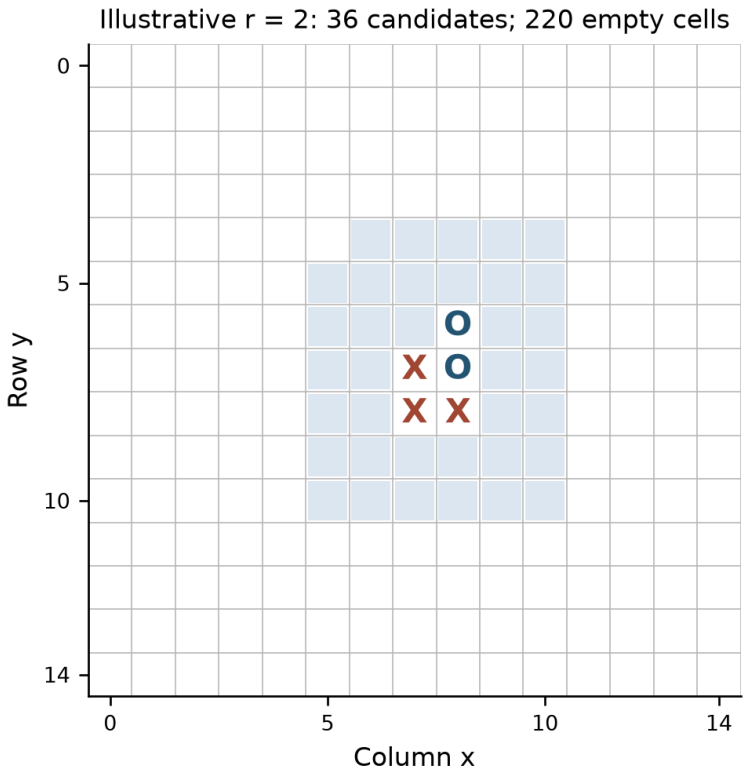


Figure 4.2. Union of clipped Chebyshev neighborhoods
Source: present/src/components/PythonVisualizer.tsx [44].

The candidate set is the union of the clipped radius-two neighborhoods around the five stones. A set removes duplicates where neighborhoods overlap, and occupied cells are excluded. The resulting count describes candidate membership for this board; the search experiments in Chapter 5 evaluate the effects of candidate policies on search work and tactical choices.

4.6.4 Empirical-analysis snippet

The fourth program calculates a win percentage and a frame-budget ratio from embedded constants, including 42.24 µs per move. It formats a pre-entered summary rather than executing a tournament or loading the battle JSON. The archived 100-game dataset reports 251.26 µs per decision; Section 5.6 distinguishes that measured value from the snippet's constant [44], [46].

The ratio uses a nominal 16.67 ms interval for 60 Hz: 42.24 µs / 16,670 µs × 100 ≈ 0.253%. This is arithmetic on the snippet's constants. Browser frame presentation also includes rendering, scheduling and communication, which are outside this calculation.

4.7 Implementation traceability and limits

The implementation can be traced from a controller action through domain state and into display, or from a search request through copied history, a Worker, a Rust/WASM object and a validated response. The critical invariants are legal board/history correspondence, move restoration during search, consistent rule interpretation, correct coordinate conversion and current-turn applicability at commit. Each has a responsible module and can be tested at its boundary rather than inferred from a single large integration result.

The application, presentation and native experiment harnesses expose the main implementation paths described in this chapter. The generated package and recorded hashes identify the binary used for the archived Node and browser runs. Appendix B maps these artifacts to their roles; Chapter 5 presents the measured results, and Chapter 6 collects remaining source-publication and validation work.

CHAPTER 5. RESULTS AND EVALUATION

5.1 Evaluation questions, evidence and boundaries

The evaluation asks four questions: whether the game and search preserve board state on the tested positions; whether Alpha-Beta agrees with Minimax on the same selected tree; how candidate selection, ordering and transposition-table reuse affect work and results; and how the actual Worker and interface behave in the recorded browser interactions. Native, Node/WASM and browser results are reported separately because they exercise different boundaries.

The application source examined for this report is tducn110/basicproject at commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. The current-engine measurements were recorded on 23 October 2026, and the separate reference measurements on 17 October 2026. The research directories contain raw CSV/JSON results, harnesses, provenance and source excerpts. A local follow-up on 12 November reproduced all non-timing fields in the 816 current-engine and 544 reference measurement rows using source files matching the archived hashes; the tables below retain the original timings [1], [24], [47].

Native experiments exercise board and search algorithms. Node/WASM parity tests load the copied binary and adapter. The headless-browser Worker experiment includes actual message transport, while the UI experiment includes React, Pixi and synthetic pointer input. Weekly reports and presentation examples supply explanatory and historical material. For each result, the discussion identifies the implementation, configuration, measurement scope and recorded environment.

5.2 Native board and search checks of the 23 October engine

The 23 October native verification used four controlled positions and sixteen seeded positions, with three timing repeats for the controlled configurations. It recorded 816 measurement rows, 408 Minimax/Alpha-Beta comparison pairs, 408 ordering-value comparison pairs and 820 root-restoration checks. The corpus also included 84 candidate configurations, 12 tactical-completion retention checks, four terminal checks and 225 native address-offset checks. All recorded assertions passed. These totals include repeated configurations; they are not counts of distinct board positions [11], [19].

Recursive search temporarily places a candidate, evaluates the child and undoes the move. The harness compares the resulting board with the saved root after each measured call. All 820 recorded restoration checks passed, and the 12 November rerun reproduced the same restoration outcomes [10], [19].

An illustrative comparison can be reconstructed directly from the published summary. For the center position at requested depth two, radius two and soft cap eight, all three configurations below return score 64 and the same best coordinate (x=6, y=6). Their search counters include the root. Their time values are the medians of three sequential native measurements, reported in nanoseconds; the timer excludes the root clone [47].

Table 5.1. Native engine search on the center fixture at depth two

Configuration	Nodes	Leaves	Cutoffs	Median time (ns)
Minimax, unordered	73	64	0	56,635
Alpha-Beta, unordered	47	38	7	41,723
Alpha-Beta, ordered	26	17	7	31,167

Relative to Minimax, unordered Alpha-Beta visits 35.62% fewer nodes: (73 − 47)/73. Ordered Alpha-Beta visits 64.38% fewer: (73 − 26)/73. Ordering reduces the Alpha-Beta count from 47 to 26, or 44.68%, and the ordered median is 44.97% below the Minimax median. Both Alpha-Beta configurations perform 264 ranking evaluations, so recursive node counts capture only part of the work. These percentages describe the selected center fixture and configuration [47].

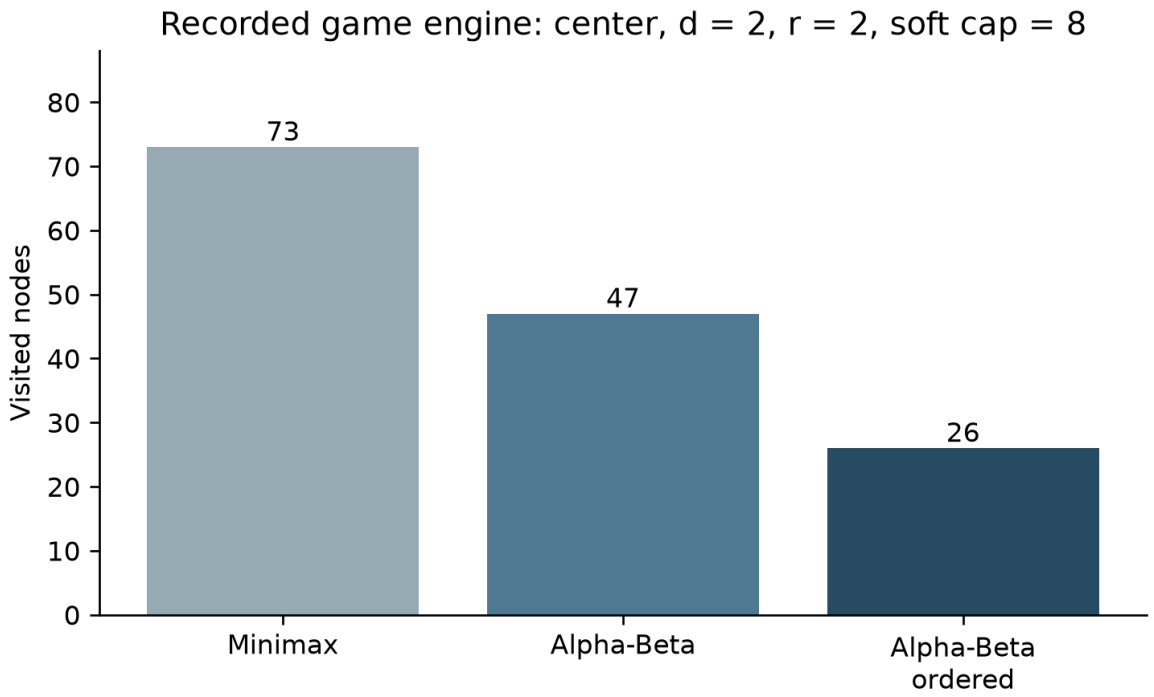


Figure 5.1. Recorded native engine nodes on the center fixture

Source: research/current_2026_10_23/engine_search_summary.csv [47].

The equality checks also have a defined meaning. When Minimax and Alpha-Beta use the same legal candidate membership, rules, evaluation and depth, agreement checks the two implementations on that selected tree. Ordering may change which equally valued move is encountered first, so a comparison that permits tied moves must compare values rather than insist on identical coordinates. A selective candidate set is a different tree from the all-legal tree; equal values on selected fixtures cannot establish that discarded moves are harmless on every board. Tactical retention checks test known immediate completions, not every future tactical sequence.

5.3 WebAssembly parity and browser observations

The Node parity run loaded the copied WebAssembly binary through synchronous initialization and used the application's executeWasmSearch adapter. Its eight multi-move fixtures contained 291 total plies. Every recorded move, board-state hash and terminal outcome agreed with the TypeScript model. The ongoing-state next player also agreed; terminal records preserve a representation difference, because Rust flips to the next player while TypeScript retains the last mover. Coordinates follow x = column, y = row and index = row × 15 + column. The binary SHA-256 was 1ff854e6493b39727d07cfcf69b0af06e349a748249d85d688741a4f85017989 [31].

The Worker run used Headless Chrome 155 on Linux and the same recorded binary hash. The fixture contained X at (row 7, column 7) and (8, 7), and O at (7, 8) and (8, 8). All six difficulty requests returned legal moves with source rust. Hard, Very Hard, Expert and Master reported depths 2, 3, 4 and 5, with 47, 252, 551 and 1,418 nodes respectively. Worker elapsed time included hydration, search and cleanup: Master recorded 3.0 ms, with about 3.3 ms round trip. Easy recorded 1.8 ms Worker elapsed and 84 ms round trip, illustrating the additional module, message and scheduling costs outside the adapter timer [32].

The UI record used a 1280 × 800 headless browser with actual React, Pixi input, DOM-dispatched PointerEvents and the Rust Worker. It exposed all six difficulty labels and displayed a depth-two result with 61 nodes and 13 cutoffs. After Undo, the details panel was removed and the Undo control was disabled; neither observation contained an alert. Controller and component tests separately assert that Undo removes a completed human–AI pair and clears its feedback [48].

The evidence therefore covers native search comparisons, six Rust Worker requests on one browser fixture and a browser move/feedback/Undo interaction. These later result records extend the earlier gate ledger, whose G3–G5 entries were still pending or partial when it was written. The evaluation uses the dated result artifacts for each observation [19], [24], [32], [48].

5.3.1 Follow-up application validation

The 27 October local validation recorded nine Vitest files and 59 passing tests, a successful TypeScript --noEmit check and a successful production build under Node 24.22.1. The same checks passed again on 12 November under Node 22.23.3, using the current application at commit 7e1e2d5 and Vite 8.3.1. The build emitted a 111.58 kB WebAssembly asset, or 34.14 kB with gzip. The separate presentation package also passed its TypeScript check and Vite 8.3.5 production build using an isolated locked dependency installation.

The 27 October browser record covers viewports of 1365 × 900 and 390 × 844. A synthetic pointer event at the canvas center was followed by a Hard response and visible search telemetry. These observations document the displayed layout and interaction at the two recorded viewport sizes.

5.4 Separate native reference experiments

The 17 October native reference experiment uses caro-core-reference to study Minimax, Alpha-Beta, candidate selection, ordering, Zobrist hashing, a transposition table and a node budget. This library and harness are separate from the

application's Rust/WASM Worker route. The local 12 November rerun used reference and harness files whose hashes matched the archived provenance and reproduced all 544 recorded rows in their non-timing fields. Appendix B identifies the corresponding artifacts [13].

The controlled fixtures use a 15 × 15 board, freestyle rules and default evaluation weights. center starts with X at (7,7) and O to move. clustered has X at (7,7) and (8,7), O at (7,8) and (8,8), and X to move. In win, X occupies (7,3) through (7,6) while O occupies (5,5) and (5,6); X can complete a five at either end. The block fixture reverses the colors, making two immediate opponent endpoints. The tactical fixtures are synthetic and need not be reachable through ordinary alternating play. Sixteen additional positions were generated through play transitions with the recorded random seed [13].

The matrix compared all legal moves and uncapped radius-one candidates at depths one and two; radius-one candidates capped at twelve at depths one through three; and radius-one candidates capped at eight at depth four. It included Minimax, Alpha-Beta with and without ordering, and optimized search with a fresh table. The dataset contains 544 measurement rows and 224 summary rows. Two additional cap-loss calls bring the verification total to 546 search/assertion calls. The harness also records 551 restorations, 112 Minimax/Alpha-Beta pairs, 112 ordering-value pairs and 208 cold-table value pairs, including repeated configurations [13].

5.4.1 A paired Minimax and Alpha-Beta example

For clustered with all legal moves, depth two and natural order, both algorithms return score 0 and best move (row 6, column 7). Minimax records 48,842 nodes, 48,620 leaves and a median of 69,252 µs. Alpha-Beta records 10,939 nodes, 10,717 leaves, 218 cutoffs and a median of 16,442 µs across three repeats. The node reduction is 77.60%, and the ratio of the recorded medians is 4.21. These figures describe the same 221-move root and depth-two tree [49].

Table 5.2. Native reference search on the clustered fixture at depth two

Clustered, all-legal, depth 2	Nodes	Leaves	Cutoffs	Median (µs)
Minimax, natural order	48,842	48,620	0	69,252
Alpha-Beta, natural order	10,939	10,717	218	16,442
Alpha-Beta, ordered	2,164	1,942	220	68,457

The ordered search visits only 2,164 nodes, but performs 48,841 extra ordering evaluations and has a median of 68,457 µs. Fewer recursively visited nodes therefore do not necessarily mean lower elapsed time, because ranking candidates also costs work. At depth one, ordering cannot prune a lower level and can add overhead. The run uses native std::time::Instant, excludes state cloning and table/Zobrist initialization from the timer, has only three sequential controlled repeats, and does not randomize order or pin a CPU. Values must not be presented as browser timings [49].

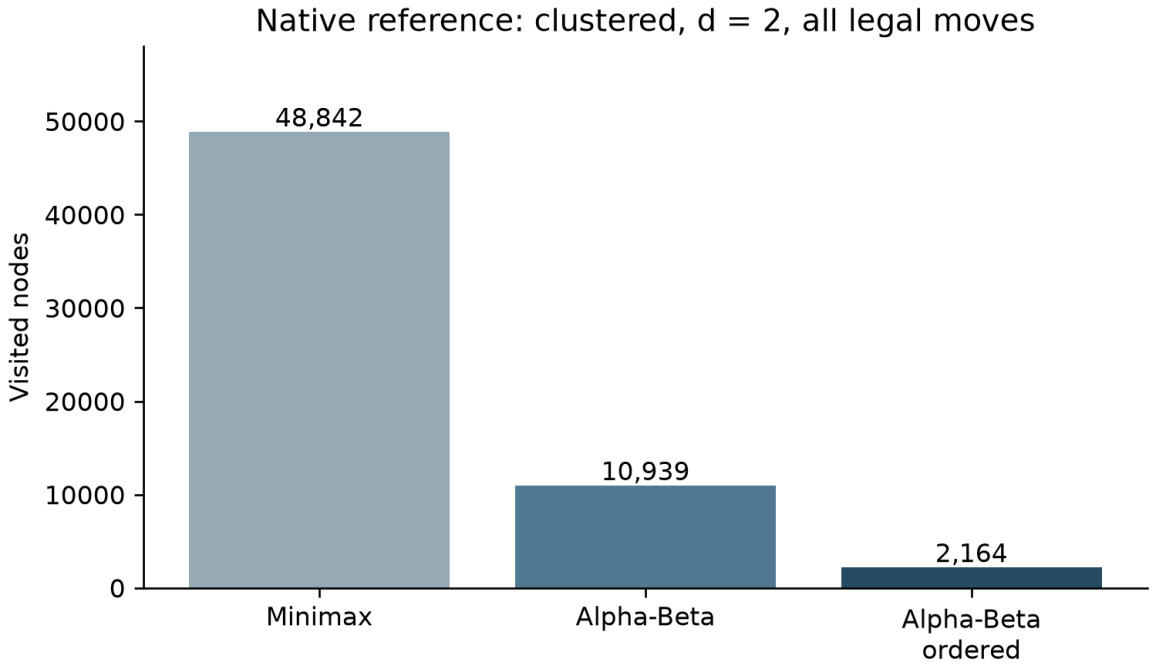


Figure 5.2. Native reference node counts (ranking cost excluded)
Source: research/week_05_09/results/search_summary.csv [49].

5.4.2 Candidate caps, ordering and tactical completeness

Candidate generation reduces the set of considered moves by selecting empty cells near existing stones. This can reduce branching but changes the search problem. The reference cap_loss.json supplies a concrete risk: in the synthetic win fixture at radius one, 20 uncapped candidates include two immediate winning endpoints (7,2) and (7,7), but a cap of six omits both. At depth one, the uncapped score is 9,999,999 and the capped score is 99,990. This is not an Alpha-Beta error: it is a difference in which moves are admitted before the search. It demonstrates why an ordering or cap policy must be evaluated against tactical membership, and why a value equality proof applies only after the move set is fixed [22].

The 23 October engine uses a different candidate policy, with a soft cap that retains immediate tactical completions. Its record includes twelve tactical-completion retention checks. The reference counterexample and the later retention checks therefore describe different policies. A controlled comparison must keep candidate membership fixed before attributing a change in work to ordering or Alpha-Beta pruning.

5.4.3 Hash, table scope and interrupted search

The reference verification record reports 3,998 forward and 3,998 undo hash checks, plus four terminal hash-axis checks. Its table semantics record reports three bound variants and observed exact, lower and upper entries. These checks concern the inspected corpus and implementation. A native entry size of 12 bytes is recorded, but the underlying hash map is not a fixed 8 MB table; reservation or native object size alone does not establish total memory consumption. The record states that rule configuration and root perspective are not included in the key and mate-ply normalization is absent. Those facts constrain safe reuse [50].

The cache-scope probe makes the constraint tangible. For a clustered fixture at radius one and cap eight, a warm table at the same root returns the same score of 0 as the initial search, with nine nodes and eight hits in the probe. After a move changes the root context, the recorded cold score is 0 and the warm score is 19,800. The record characterizes cross-root reuse of root-relative entries as unsupported. A conservative implementation must start with a fresh table or identify all evaluation and rule context in its namespace; the example does not prove that every warm reuse is unsafe [27].

The budget artifact tests restoration during interruption. A direct budget of ten is exhausted and restores the root. In the iterative case, a budget of fifty is exhausted after a completed depth of two; 40 aggregate nodes are reported, and the record explicitly says that aggregate excludes the aborted iteration. It is therefore incorrect to describe 40 as the complete work consumed by all attempted iterations. The last completed result remains meaningful only when incomplete iteration state is not presented as a completed deeper result [51].

5.5 Historical cross-language and Week 4 datasets

A historical benchmark compares a native Rust reference harness with a TypeScript harness running in Node on one midgame fixture at depths one through four. At depth two, both record score 19,990, 609 nodes and 39 cutoffs; the times are 0.3125 ms for Rust and 5.5795 ms for TypeScript. At depth four, both record score

9,999,997, 496,790 nodes and 16,350 cutoffs, with times of 285.4517 ms and 5,083.4518 ms. The matching scores and counters establish agreement for these recorded runs [52].

Table 5.3. Historical native Rust and TypeScript/Node benchmark

Depth	Nodes, each row	Rust native (ms)	TypeScript/Node (ms)
1	45	0.0221	0.5383
2	609	0.3125	5.5795
3	16,184	9.0248	156.7846
4	496,790	285.4517	5,083.4518

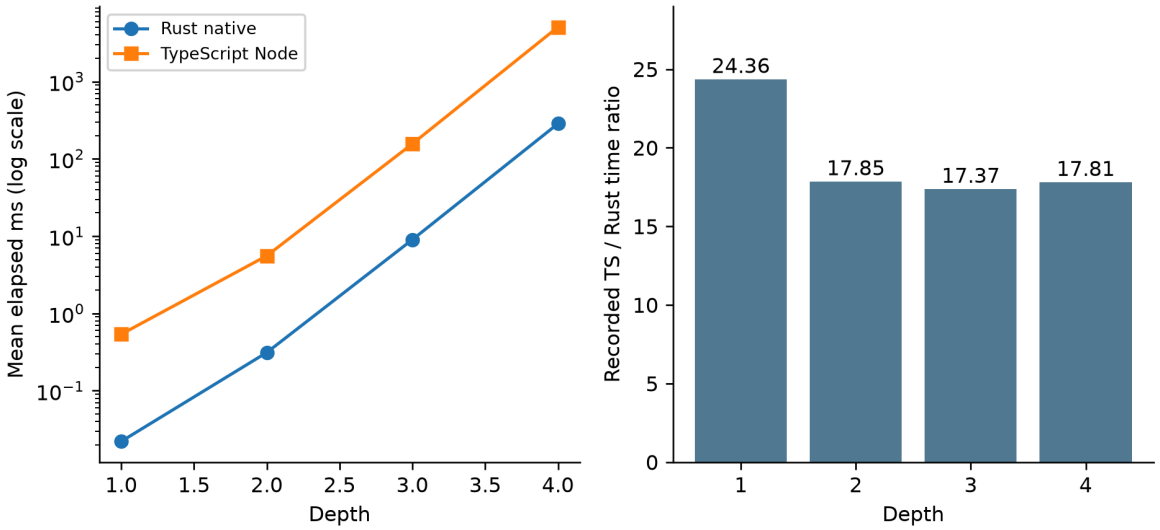


Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture
Source: benchmark/results/benchmark_data.json [52].

Across the four depths, the recorded Node-to-native time ratios range from 17.4 to 24.4. Both harnesses report matching node counts, cutoff counts and scores on this fixture. The comparison describes these two implementations and runtime environments; the application's WebAssembly execution is evaluated separately in Section 5.3 [52].

The Week 4 native battle dataset records 100 Rule-Based wins against Random, zero Random wins and zero draws, averaging 9.90 plies per game. It reports 251.26 µs per decision, 220.21 empty cells and 60.80 candidates on average. The ratio of

the two mean cell counts gives a 72.39% reduction in candidate membership. Seven separate pattern fixtures are marked PASS in the same dataset [46].

5.6 Presentation examples and evidence quality

The presentation application combines explanatory components, a figure inspector and a Python visualizer. Its horizontal pattern scanner, demonstration weights and five-stone Chebyshev example illustrate distinct parts of the algorithms. Their inputs and outputs are described in Section 4.6 and remain separate from the game Worker measurements [44].

The empirical-analysis snippet contains the constant 42.24 µs, whereas the battle JSON records 251.26 µs per decision. The report uses the dataset value for the Week 4 result. Ratios against a nominal 16.67 ms interval explain units and budgets; they do not measure browser frame presentation. Figures derived from these constants are treated as instructional illustrations [44], [46], [52].

5.7 Threats to validity and interpretation

Internal validity. Search comparisons require the same position, depth, terminal rules, evaluation perspective and candidate set. Otherwise an apparent improvement can arise from searching a different tree. A table that ignores rule or perspective context can return a stale value. An aborted iteration must be counted separately from completed work. These issues are directly motivated by the cap-loss, cache-scope and budget probes.

Measurement validity. The native experiments distinguish recursive nodes, leaf evaluations, ranking evaluations, cutoffs and time. Treating one counter as a substitute for all computation can reverse the apparent conclusion, as the clustered ordered example shows. Three sequential timing repeats and lack of randomized order, CPU isolation or uncertainty intervals support descriptive comparisons only. Native timer scope, Worker hydration and search, and browser round-trip duration are different quantities.

External validity. The fixtures cover controlled and seeded boards but not the full space of legal games. Tactical test positions may be synthetic. A 100-game Rule-Based versus Random result is narrow, and it does not calibrate six difficulty levels against human skill. Node execution and headless-browser synthetic input do not establish physical touchscreen, Wink host, deployment or production behavior.

Reproducibility and provenance. The repository supplies raw measurements, harnesses, source excerpts and binary hashes. The 12 November follow-up matched 36 available engine, reference, harness and lock files to their archived hashes and

reproduced the non-timing search results. On 13 November, the modified engine was included in the local application source package and rebuilt to the same WebAssembly bytes. The native reference library remains a separately identified dependency. Appendix B records these checks without replacing the dates or environments of the historical experiments.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Work completed and contribution

This project studies how board representation, Freestyle win detection, heuristic evaluation and adversarial search solve the move-selection problem in Caro. A web application connects those decisions to a React interface and PixiJS board through a Rust/WebAssembly search Worker, providing a concrete setting for checking legal state transitions and asynchronous result handling. A separate teaching website provides visual explanations and executable Python examples. Weekly reports and native research artifacts document the algorithmic development and experimental evaluation [1], [53].

The main contribution is an implementation-level account of how game decisions depend on representation and search policy. Board updates, history and rollback support recursive exploration. Terminal utility takes precedence over the static heuristic. Minimax defines alternating decisions, and Alpha-Beta pruning preserves the value of the same selected search tree while omitting branches that cannot change it. Candidate filtering changes the tree itself; ordering changes the sequence in which its moves are examined. The reference experiments extend this analysis to Zobrist hashing, transposition-table bounds and interrupted iterative search [11], [13].

The recorded native engine experiment reports 408 Minimax/Alpha-Beta comparisons with matching values and 820 successful restoration checks. Separate adapter and browser records exercise the connection between search and the application. In the reference experiments, move ordering reduced visits on some positions, but its evaluation work could outweigh the time saved. A hard cap discarded immediate wins in one fixture, and cache reuse after a root-player change returned an incorrect score in another. These results show that depth and node count alone do not assess correctness or efficiency [19], [22], [27], [31], [32], [49].

The final local application provides all six report profiles: Easy, Normal, Hard, Very Hard, Expert and Master. A fresh build from the included Rust source reproduced the existing application binary byte-for-byte. Browser smoke checks selected each profile and observed a response with completed depths 0, 1, 2, 3, 4 and 5 on the checked opening. The presentation now embeds this local application. These checks establish the tested configuration and interaction; playing-strength calibration remains future work.

6.2 Limitations

The active search is selective. Its soft tactical cap retains immediate winning and blocking completions, but deeper positional moves remain subject to the candidate limit. The configured limit falls from 15 at Hard to 8 at Master as depth rises from two to five plies. The static evaluator scores contiguous runs; gapped formations and intersecting threats depend on tactical assessment and deeper search. Persistent transposition tables and iterative-deepening budgets are studied in separate implementations rather than used by the active Rust search [10], [13], [29].

The implementation review also identifies boundary cases for further correction. GameController starts a round before validating the first direct move request, and the core coordinate predicate checks bounds without requiring integer indices. The normal pointer path supplies integer cells, while Worker responses have a separate integer check. Drag detection compares consecutive pointer positions, which can miss a slow drag made of small steps. In the teaching website, a failed or missing Pyodide loader can leave initialization state or waiting callers unresolved [2], [35], [37], [43].

The measurements cover a finite corpus and specific native or browser environments. The published Git revision includes the application and binary but omits the complete modified engine and caro-core-reference dependency. The local release now supplies the modified engine and a reproducible application build; reproducing the separate reference experiments still requires their dependency sources. Historical timing and bot-match summaries retain narrower provenance than the controlled research records. Physical-device interaction, live host integration, human usability and calibrated playing strength remain subjects for further evaluation [1], [11], [13], [24], [46], [52].

6.3 Future development

Release maintenance should preserve the source-to-binary connection established by the local rebuild. The modified engine, lockfile, build command and generated-package hashes should be published together. The separate reference library and experiment harnesses should also be distributed with their exact dependencies. Each new experiment should identify its source revision, toolchain, platform and binary hash, while the earlier baseline remains available under its own identity.

Input validation and lifecycle handling should be strengthened at their owning boundaries. The core should require integer coordinates and validate a move before advancing the match phase. Gesture classification should use displacement from the

initial press, and the Pyodide loader should settle all waiting requests and permit retry after failure. Regression cases should include invalid first moves, fractional inputs, slow drags and failed runtime initialization.

Further search evaluation should separate candidate selection from move ordering. A fixed corpus of reachable positions can compare all-legal search with radius and cap policies, including known wins, blocks and intersecting threats. Useful outputs include missed tactics, chosen moves, root values, ranking evaluations, recursive visits and elapsed time. Multiple seeds and explicit tie handling would make the comparison less dependent on one fixture.

Persistent caching requires a defined compatibility context. Future work should include rules, evaluation weights, root perspective and mate-distance treatment in the cache design, then test exact, lower and upper bounds across depths and search windows. Warm-table and cold-table comparisons should include root changes and collisions. Memory measurements should include map capacity and allocation overhead as well as entry size.

An anytime search could combine iterative deepening with cancellation and an explicit last-completed-result policy. Its telemetry should distinguish requested depth from completed depth and count work spent in aborted iterations. Tests should cover Reset, Undo, stale replies and Worker failure while confirming that interrupted search restores the input position.

Application evaluation should extend to physical target devices and the intended host environment. Repeated measurements of response latency and frame timing would complement the present automated checks. A participant study could examine control usability and perceived difficulty, while a multi-seed tournament against stated opponents could assess playing strength. Both require a documented protocol before their results can support conclusions.

The teaching website should remain synchronized with the implementations it explains. Illustrative programs need explicit scope, especially where a horizontal example accompanies a four-direction rule description. Charts should link to the data and generation procedure that produced them. This would let readers move from an explanation to the corresponding source or experiment without confusing a demonstration with a measurement.

REFERENCES

[1] tducn110, “Repository snapshot,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc

[2] tducn110, “CaroGame/src/app/GameController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/app/GameController.ts

[3] tducn110, “CaroGame/src/game/ai/AIController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/AIController.ts

[4] tducn110, “CaroGame/src/App.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/App.tsx

[5] tducn110, “CaroGame/src/pixi/CaroScene.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/pixi/CaroScene.ts

[6] tducn110, “CaroGame/src/game/wasm/GomokuWasmEngine.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/GomokuWasmEngine.ts

[7] tducn110, “CaroGame/src/game/wasm/worker.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/worker.ts

[8] tducn110, “CaroGame/src/game/core/WinRules.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/WinRules.ts

[9] tducn110, “research/current_2026_10_23/baseline_engine/src/board.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/board.rs

[10] tducn110, “research/current_2026_10_23/SOURCE_EXCERPTS.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/SOURCE_EXCERPTS.md

[11] tducn110, “research/current_2026_10_23/README.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/README.md

[12] tducn110, “research/current_2026_10_23/baseline_engine/src/ai.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/ai.rs

[13] tducn110, “research/week_05_09/README.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/README.md

[14] tducn110, “CaroGame/src/game/wasm/contracts.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/contracts.ts

[15] tducn110, “CaroGame/src/game/bot/search.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/search.ts

[16] C. E. Shannon, “Programming a computer for playing chess,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, ser. 7, vol. 41, no. 314, pp. 256–275, Mar. 1950, doi: 10.1080/14786445008521796. [Online]. Available: https://doi.org/10.1080/14786445008521796

[17] The Rust Project Developers, “Type layout,” The Rust Reference. Accessed: Nov. 12, 2026. [Online]. Available: https://doc.rust-lang.org/reference/type-layout.html

[18] tducn110, “CaroGame/src/game/core/GameState.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/GameState.ts

[19] tducn110, “research/current_2026_10_23/engine_verification.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_verification.json

[20] tducn110, “CaroGame/src/game/ai/TacticalMoveResolver.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/TacticalMoveResolver.ts

[21] tducn110, “research/current_2026_10_23/baseline_engine/src/eval.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/eval.rs

[22] tducn110, “research/week_05_09/results/cap_loss.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cap_loss.json

[23] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975, doi: 10.1016/0004-3702(75)90019-3. [Online]. Available: https://doi.org/10.1016/0004-3702(75)90019-3

[24] tducn110, “research/current_2026_10_23/GATES.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/GATES.md

[25] A. L. Zobrist, “A new hashing method with application for game playing,” *Computer Sciences Department*, University of Wisconsin–Madison, Madison, WI, USA, Tech. Rep. 88, Apr. 1970. Accessed: Nov. 12, 2026. [Online]. Available: https://research.cs.wisc.edu/techreports/1970/TR88.pdf

[26] tducn110, “CaroGame/src/game/bot/board.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/board.ts

[27] tducn110, “research/week_05_09/results/cache_scope_probe.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cache_scope_probe.json

[28] tducn110, “CaroGame/src/game/bot/eval.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/eval.ts

[29] tducn110, “research/current_2026_10_23/engine_difficulty_runs.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_difficulty_runs.csv

[30] tducn110, “CaroGame/src/game/wasm/executeSearch.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/executeSearch.ts

[31] tducn110, “research/current_2026_10_23/wasm_ts_parity.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/wasm_ts_parity.json

[32] tducn110, “research/current_2026_10_23/G4_real_browser_worker.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G4_real_browser_worker.json

[33] tducn110, “CaroGame/src/game/behaviors/match/MatchController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/match/MatchController.ts

[34] WHATWG, “Web workers,” HTML Living Standard. Accessed: Nov. 12, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/workers.html

[35] tducn110, “CaroGame/src/board/spatial/CoordinateTransform.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/spatial/CoordinateTransform.ts

[36] tducn110, “CaroGame/src/game/behaviors/move/PlayMove.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/move/PlayMove.ts

[37] tducn110, “CaroGame/src/board/input/BoardInputController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/input/BoardInputController.ts

[38] tducn110, “CaroGame/package.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/package.json

[39] tducn110, “present/src/App.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/App.tsx

[40] tducn110, “CaroGame/src/game/wasm/engine,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/engine

[41] tducn110, “CaroGame/src/i18n.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/i18n.ts

[42] tducn110, “present/src/utils/caroEngine.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/caroEngine.ts

[43] tducn110, “present/src/utils/pyodideRunner.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/pyodideRunner.ts

[44] tducn110, “present/src/components/PythonVisualizer.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/components/PythonVisualizer.tsx

[45] WebAssembly Community Group, “WebAssembly Specification,” release 3.0, Nov. 4, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://webassembly.github.io/spec/core/

[46] tducn110, “benchmark/results/week4_bot_battle.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/week4_bot_battle.json

[47] tducn110, “research/current_2026_10_23/engine_search_summary.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_search_summary.csv

[48] tducn110, “research/current_2026_10_23/G5_browser_ui.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G5_browser_ui.json

[49] tducn110, “research/week_05_09/results/search_summary.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/search_summary.csv

[50] tducn110, “research/week_05_09/results/tt_semantics.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/tt_semantics.json

[51] tducn110, “research/week_05_09/results/budget_abort.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/budget_abort.json

[52] tducn110, “benchmark/results/benchmark_data.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/benchmark_data.json

[53] tducn110, “Week_05_09_Report_Index.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. Accessed: Nov. 12, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/Week_05_09_Report_Index.md

APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP

Table A.1 inventories the 20 top-level directories in the pinned Git tree and groups the top-level files in one additional row. The inventory describes the published repository; Git excludes empty directories and local untracked files. Weekly reports provide the project chronology, while dated source and experimental artifacts identify the implementation associated with a technical result [1], [53].

Table A.1. Repository inventory and roles

Item	Role and interpretation
.gemini	Repository configuration and project working-context files.
.obsidian	Obsidian vault configuration and navigation files.
CaroGame	Playable application, interface components, board rendering, WebAssembly package, assets and tests.
Week1	Project proposal, initial scope and early diagrams.
Week2	Board representation and game-rule foundations.
Week3	UI and component architecture material and diagrams.
Week4	Pattern, heuristic, Random and Rule-Based material, including historical figures.
Week5	Minimax and search-tree analysis documents.
Week6	Alpha-Beta pruning and related search documentation.
Week7	Candidate-generation documentation and its selected-move trade-off.
Week8	Move-ordering documentation and search-order trade-off.
Week9	Zobrist hashing and transposition-table documentation.
Week10	Iterative deepening, budgets and evaluation documentation.
archived	Earlier report drafts were retained as project history.
assets	Shared figures, screenshots and charts.

Item	Role and interpretation
benchmark	Rust and TypeScript benchmark harnesses and JSON summaries comparing native Rust and Node execution across depths 1–4.
present	Presentation application, components, assets, static data and Python examples.
research	Dated engine evidence, baseline/excerpts, native research harness, raw CSV and JSON artifacts.
rules	Repository working rules and layout guidance.
scripts	Data/diagram-generation and benchmark orchestration scripts.
Top-level files	Repository overview, layout and planning documents, review reports, benchmark discussion, report index, configuration and report template.

The weekly documentation follows the development topics. Weeks 1–3 cover the problem, board and interface; Week 4 covers local patterns and baseline bots; Weeks 5–6 examine adversarial search and pruning; Weeks 7–8 examine candidate selection and ordering; Weeks 9–10 address hashing, table reuse and interrupted search. The dated research records give the experimental context for these topics [13], [53].

The ten-week documentation sequence is summarized below. Each entry names the topic covered in the corresponding weekly material.

W01: Problem definition, actors, board model and project scope.
W02: Coordinates, board updates, legal moves and terminal-state rules.
W03: Interface components, application ownership and match lifecycle.
W04: Local run patterns, heuristic evaluation, Random and Rule-Based baselines.
W05: Minimax recurrence, search depth, recursive state updates and rollback.
W06: Alpha-Beta bounds, cutoffs and comparison with Minimax.
W07: Candidate neighborhoods, selective search and difficulty configuration.
W08: Move ordering, search visits and the cost of ranking candidates.
W09: Zobrist hashing, incremental updates and transposition-table bounds.
W10: Iterative deepening, search interruption and evaluation.

APPENDIX B. EVIDENCE AND VERIFICATION REGISTER

Table B.1. Evidence summary and verification status

Evidence or claim	What the snapshot contains	Recorded result and scope
23 October modified engine	Selected source excerpts, hashes, raw measurements and verification records.	engine_verification.json reports 816 search rows, 408 Minimax/Alpha-Beta pairs and 820 restoration checks in native execution [19].
23 October app modifications	Application source, tests and the vendored WebAssembly package are present in CaroGame.	The package defines Vitest tests, TypeScript checking and a Vite build; results must be associated with the revision and environment of the test run [38].
Native reference library	Harness, results, detailed source hashes and configuration.	The reference records report 544 timed search rows, 112 Minimax/Alpha-Beta comparisons, 551 root restorations and 3,998 forward plus 3,998 undo hash checks [13].
Binary and adapter parity	Finite Node parity record and copied WASM hash; separate real browser Worker record.	The Node record covers 8 histories and 291 plies. A separate headless Chrome Worker record exercises all 6 difficulty configurations on one clustered position [31], [32].
Browser UI	Headless browser records of synthetic pointer input, Undo and viewport layout.	The UI record exercises one move/Undo sequence at 1280 × 800. The viewport artifact records layout at 1280 × 800, 390 × 844 and 320 × 568 [48].

Evidence or claim	What the snapshot contains	Recorded result and scope
12 November revalidation	Current frontend and native test logs; rerun search outputs compared with the archived non-timing fields.	The review rerun passed 59 frontend tests in 9 files, typecheck and the Vite 8.3.1 build under Node 22.23.3/npm 10.9.9. Native tests passed: 38 unit and 7 integration tests for the current engine, and 6 unit and 3 integration tests for the reference library, using rustc 1.93.1. All 816 engine rows and 544 reference rows reproduced the archived decisions and counters; timing fields were excluded from this comparison.
Gate status	An early gate ledger and later native, adapter, Worker and interface artifacts.	The ledger lists G3 as pending, G4 as partial and G5 as pending. Later files supply specific observations; they do not constitute an updated sign-off for every gate [19], [24], [31], [32], [48].
Benchmarks	Raw or summary files and stated fixture/measurement scope.	The historical summaries report Node-to-native Rust time ratios of 17.4–24.4 on one midgame fixture and 100 Rule-Based wins in 100 matches against Random. These are dataset-specific results [46], [52].
Presentation	Source code, figures and embedded statistics.	The source provides React/Vite explanations, KaTeX formulas, a figure inspector and four Pyodide program definitions [39], [43], [44].

13 November local release verification. Eleven Rust source, manifest, test and example files were copied unchanged into CaroGame/engine. A locked offline wasm-pack release build, with wasm-opt disabled, regenerated the five runtime files. All five matched the previous canonical package. The WASM file is 111,582 bytes with SHA-256

1ff854e6493b39727d07cfcf69b0af06e349a748249d85d688741a4f85017989. The source and output manifest contains 16 checked hashes.

Native tests passed 38 unit and 7 integration cases. The frontend passed 65 tests in 9 files, typecheck and the Vite build. The runtime and contract subset contains 28 tests, including six paired immediate-win/forced-block cases on the actual WASM package; Easy is checked for legal Random behavior, while the other five profiles take the unique tactical completion. Browser smoke checks selected each tier and observed completed depths 0/1/2/3/4/5 after the checked opening move. This is finite configuration and interaction evidence, without a tournament-strength conclusion.

The presentation was built after its playable entry points were switched to the local application. The previous public deployment served four selectors and a different 48,549-byte WASM package; the local release does not replace that deployment. The rebuild and application checks retain their own evidence files: ENGINE_MIGRATION_SOURCE_PARITY.json, WASM_REBUILD_RUNTIME_EVIDENCE.json, ENGINE_MIGRATION_VERIFICATION.md and src/game/wasm/engine/provenance.json.

APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE

The first procedure inspects the published revision and runs its frontend scripts. The local release described later in this appendix additionally contains CaroGame/engine and the rebuild script. Its changes are supplied in the local source package; checking out the published SHA alone does not retrieve them. Native research records retain their original harnesses, manifests, results and provenance [1], [11], [13].

```
git clone https://github.com/tducn110/basicproject.git
cd basicproject
git checkout 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc
git status --short
```

From the repository root, install the game dependencies from its lockfile, then run the declared typecheck, test and build scripts. Use a Node version compatible with the locked packages and record node --version and npm --version with the results.

```
cd CaroGame
npm ci
npm run typecheck
npm test
npm run build
```

In a separate terminal at the repository root, install and build the presentation website. The development command starts its interactive preview.

```
cd present
npm ci
npm run build
npm run dev
```

To inspect the native experiment provenance from the repository root, read the manifests and recorded results below. These commands do not regenerate the datasets.

```
cat research/week_05_09/Cargo.toml
research/week_05_09/results/provenance.json
cat research/current_2026_10_23/provenance.json
git ls-tree -d --name-only HEAD
```

The original reference runner requires caro-core-reference at the repository root. The current-engine research runner expects gomoku-main/engine and an application directory named 10_caro; its WASM stage copies generated files into that application. These paths belong to the original experiment workspace. The local application release instead owns its Rust source at CaroGame/engine and uses the following commands from CaroGame to rebuild it. Rust with the

wasm32-unknown-unknown target and wasm-pack must be installed; the offline build also requires the locked Cargo dependencies in the local cache.

npm run build:wasm

cargo test --manifest-path engine/Cargo.toml --locked --offline

npm run typecheck

npm test

npm run build

The build writes src/game/wasm/engine/provenance.json with the input hashes, output hashes and tool versions. The 13 November build used rustc/cargo 1.98.1 and wasm-pack 0.15.0. The current engine toolchain selects stable, so a later build must record the resolved version and compare its generated hashes rather than assume identical bytes.

For the local defense demonstration, start the game with npm run dev from CaroGame, then start npm run dev from present in another terminal. The game uses port 8443 and the presentation uses port 5174. Local presentation entry points use the game on the same host at port 8443; VITE_CARO_GAME_URL can explicitly select another application endpoint. Open http://localhost:5174 for the presentation or http://localhost:8443 for the game.

The local release contains all six difficulty selectors and uses the rebuilt Worker/WASM route. The external 10-caro.vercel.app deployment was not updated in this local release and should not be used to demonstrate its six-tier contract. Regenerating the separate native reference datasets still requires their exact dependency sources and original harness layout [11], [13].